113 energy-wasting code patterns, measured

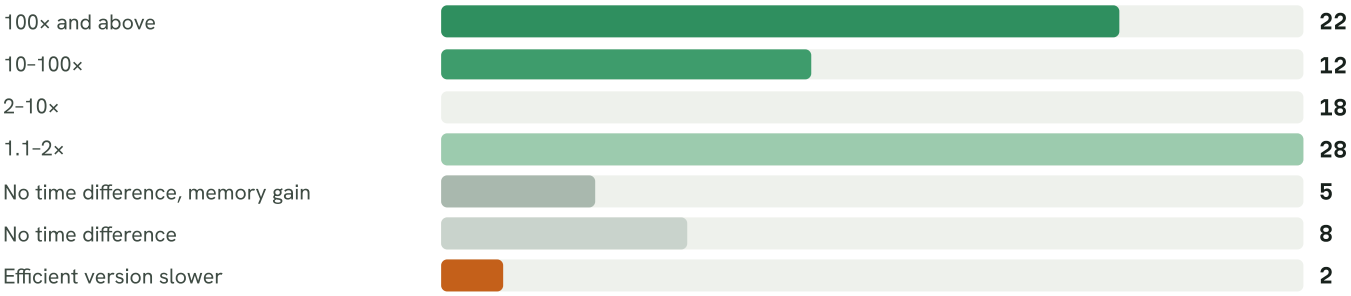
Green Coding and Digital Carbon Footprint Optimisation Tool
Sakarya University · Yunus Emre Sevindik · Mehmet Zahid Görgeç

SUMMARY

For each of the **113 patterns** Yeşil Kod looks for, a wasteful and an efficient example were written. Both ran on the same input; their outputs were compared and their time and memory measured. **95 rules were measured**; in all 94 whose output could be compared, both versions gave exactly the same result, and 1 has a random output. 18 rules need a database server, a GPU or the network, so they were not measured; the reasons are listed. The engine fixes **11 patterns** by itself; its fixes were tested the same way.

2.83×
median measured gain

MEASURED GAIN (wasteful version's time ÷ efficient version's time)



How to read it

- **Severity**: *high* waste that multiplies as data grows, *medium* clear waste, *info* small but easy gain.
- **Engine fixes it**: Yeşil Kod rewrites this pattern in the code by itself; for the others it gives advice.
- **Gain**: the wasteful version's time divided by the efficient version's (median of 5 rounds). For waiting rules the time is the same, so the ratio comes from CPU time.
- Results showing **no difference** or **slower** are listed too; the reason is in the note under each rule.
- Highlighted lines in the code are the lines that differ between the two versions.

Measured on Apple M4, Python 3.13.9, 29 October 2026. Estimated energy is proportional to CPU time; on the same processor the ratio equals the energy ratio.

Open any rule on your phone and measure it on your own device.

yes3760.github.io/yesil-kod/?lang=en#kurallar

Groups

01 Database 14	02 Loops and algorithms 18	03 Data structures and Python idioms 31
04 Text and regular expressions 7	05 Data science (pandas, NumPy) 17	06 Machine learning 4
07 Network and cloud 7	08 Files and I/O 8	09 Concurrency and waiting 7

01 Database

14 rules

YK001 high Engine fixes it — 1.86× faster

Database query inside a loop (N+1)

Every iteration sends its own request to the database. Fetch everything with one JOIN or a WHERE id IN (...) query.

WHY DOES IT WASTE ENERGY?
Every query is a network round trip, a parse and a disk access. A thousand separate queries for a thousand rows cost far more CPU, network and waiting than fetching the same data in one query.

WHEN IS IT FINE?
If the loop runs only a few times, or every query goes to a different database, the difference is small.

WORK DONE `n queries → 1 query`

WASTEFUL

```
def siparis_raporu(db, siparisler):
    rapor = []
    for s in siparisler:
        ad = db.execute("SELECT ad FROM musteri WHERE id = ?",
  (s["musteri_id"],)).fetchone()[0]
        rapor.append((s["id"], ad))
    return rapor
```

EFFICIENT

```
def siparis_raporu(db, siparisler):
    kimlikler = sorted({s["musteri_id"] for s in siparisler})
    yer_tutucular = ", ".join("?" * len(kimlikler))
    sorgu = "SELECT id, ad FROM musteri WHERE id IN (" +
  yer_tutucular + ")"
    adlar = dict(db.execute(sorgu, kimlikler).fetchall())
    return [(s["id"], adlar[s["musteri_id"]]) for s in
  siparisler]
```

MEASURED time 2.1 ms → 1.1 ms · memory 121.4 KB → 280.5 KB · same result ✓

Measured on in-memory SQLite. On a database across the network every query also adds a round trip (usually 0.2–1 ms), so the gap grows much larger. The engine's fix gave the same result too (1.42×).

Source: Green Software Foundation: Keep request counts low · Django: select_related · `docs.djangoproject.com/en/stable/topics/db/optimization/`

YK021 info Gives advice — 3.80× faster

Fetching needless columns with SELECT *

SELECT * also reads unused columns from disk and carries them over the network. List only the columns you need.

WHY DOES IT WASTE ENERGY?
Unused columns are also read from disk, held in memory and sent over the network; on wide tables that is several times the needed I/O. It also prevents covering-index use.

WHEN IS IT FINE?
If every column is really used, or the table is very narrow, the difference is small.

WORK DONE `every column → needed columns`

WASTEFUL

```
def musteri_adlari(db):
    return [r[1] for r in db.execute("SELECT * FROM musteri")]
```

EFFICIENT

```
def musteri_adlari(db):
    return [r[0] for r in db.execute("SELECT ad FROM musteri")]
```

MEASURED time 2.8 ms → 736 µs · memory 150.6 KB → 148.6 KB · same result ✓

Source: Green Software Foundation: Reduce transmitted data · Django: only()/values() · `patterns.greensoftware.foundation/development`

YK022 medium Gives advice — 1,201× faster

Fetching every row just to count

len(...fetchall()) carries every row out of the database. Let the database count with SELECT COUNT(*).

WHY DOES IT WASTE ENERGY?
Fetching every row to count means moving them over the network and turning them into Python objects. COUNT(*) returns one number, often computed from an index.

WHEN IS IT FINE?
If the rows are fetched for other work anyway, len() adds nothing.

WORK DONE `moving n rows → 1 number`

WASTEFUL

```
def siparis_sayisi(db):
    return len(db.execute("SELECT id FROM siparis").fetchall())
```

EFFICIENT

```
def siparis_sayisi(db):
    return db.execute("SELECT COUNT(*) FROM
  siparis").fetchone()[0]
```

MEASURED time 5.1 ms → 4.3 µs · memory 1.6 MB → 673 B · same result ✓

Source: Django: Database access optimization (count) · `docs.djangoproject.com/en/stable/ref/models/querysets/#count`

YK023 medium Gives advice 1.91× faster

Filtering in Python instead of the database

Fetching every row and discarding most in Python moves needless data. Put the condition in the WHERE clause so the database can use its indexes.

WHY DOES IT WASTE ENERGY?

Unneeded rows are read from disk, moved over the network and dropped in Python. With WHERE the database uses its indexes and returns only what is needed.

WHEN IS IT FINE?

If the condition cannot be expressed in SQL (complex Python logic), filtering in Python may be necessary.

WORK DONE `n rows → matching rows`

WASTEFUL

```
def buyuk_siparisler(db):
    return [s for s in db.execute("SELECT id, tutar FROM
  siparis").fetchall() if s[1] > 500]
```

EFFICIENT

```
def buyuk_siparisler(db):
    return db.execute("SELECT id, tutar FROM siparis WHERE
  tutar > ?", (500,)).fetchall()
```

MEASURED time 8.1 ms → 4.2 ms · memory 2.3 MB → 1.1 MB · same result ✓

Source: Green Software Foundation: Reduce transmitted data · Django: filter() · `docs.djangoproject.com/en/stable/topics/db/optimization/`

YK031 high Gives advice 143× faster

Commit inside a loop

Every commit syncs the transaction log to disk (fsync); committing in a loop means hundreds of disk waits. Commit once after the loop.

WHY DOES IT WASTE ENERGY?

Every commit waits for the transaction log to be written durably (fsync); the disk's physical write time is multiplied by the number of iterations. The SQLite FAQ names this as the main reason INSERTs are slow.

WHEN IS IT FINE?

If each record must be durable on its own (payments), separate commits are required.

WORK DONE `n disk syncs → 1`

WASTEFUL

```
def kayitlari_ekle(baglanti, kayitlar):
    for k in kayitlar:
        baglanti.execute("INSERT INTO log (mesaj) VALUES (?)",
  (k,))
        baglanti.commit()
```

EFFICIENT

```
def kayitlari_ekle(baglanti, kayitlar):
    with baglanti:
        baglanti.executemany("INSERT INTO log (mesaj) VALUES
  (?)", [(k,) for k in kayitlar])
```

MEASURED time 60.8 ms → 425 µs · memory 19.8 KB → 4.0 KB · same result ✓

Measured with an SQLite file on disk.

Source: SQLite SSS: INSERT neden yavaş (işlem başına disk eşitleme) · `www.sqlite.org/faq.html#q19`

YK032 high Gives advice 1.57× faster

Row-by-row INSERT, UPDATE or DELETE in a loop

A separate write per row is a separate round trip to the database. Use executemany() or a bulk insert.

WHY DOES IT WASTE ENERGY?

Every execute is a separate round trip, parse and planning step; executemany runs the same prepared statement in bulk.

WHEN IS IT FINE?

With very few rows the difference is negligible.

WORK DONE `n round trips → 1`

WASTEFUL

```
def fiyat_guncelle(db, urunler):
    for u in urunler:
        db.execute("UPDATE urun SET fiyat = ? WHERE id = ?",
  (u["fiyat"], u["id"]))
```

EFFICIENT

```
def fiyat_guncelle(db, urunler):
    db.executemany("UPDATE urun SET fiyat = ? WHERE id = ?",
  [(u["fiyat"], u["id"]) for u in urunler])
```

MEASURED time 2.8 ms → 1.8 ms · memory 18.1 KB → 80.7 KB · same result ✓

Source: PEP 249 (DB-API): executemany · `peps.python.org/pep-0249/#executemany`

YK033 high Gives advice not measured

objects.create() in a loop (Django)

Every create() is a separate INSERT. Collect the objects and insert them with bulk_create() in one query.

WHY DOES IT WASTE ENERGY?

Every create() is a separate INSERT, often in its own transaction; bulk_create inserts in one query.

WHEN IS IT FINE?

If you rely on post_save signals or custom save() logic, bulk_create does not run them.

WORK DONE **n queries → 1**

WASTEFUL

```
def ogrenci_kaydet(satirlar):
    for s in satirlar:
        Ogrenci.objects.create(ad=s["ad"], numara=s["no"])
```

EFFICIENT

```
def ogrenci_kaydet(satirlar):
    Ogrenci.objects.bulk_create([Ogrenci(ad=s["ad"],
  numara=s["no"]) for s in satirlar])
```

NOT MEASURED Needs a Django project and a database server. The gain is in query count: n INSERTs → 1.

Source: Django belgeleri: bulk_create · docs.djangoproject.com/en/stable/ref/models/querysets/#bulk-create

---

YK034 medium Gives advice not measured

save() inside a loop

Saving each object in a loop runs a separate UPDATE every time. Use bulk_update() or queryset.update().

WHY DOES IT WASTE ENERGY?

Every save() sends a separate UPDATE with every field.

WHEN IS IT FINE?

If save() holds custom logic or signals, a bulk update skips them.

WORK DONE **n queries → 1**

WASTEFUL

```
def notlari_yukselt(ogrenciler):
    for o in ogrenciler:
        o.not_ortalamasi += 5
        o.save()
```

EFFICIENT

```
from django.db.models import F


def notlari_yukselt(ogrenciler):
    ogrenciler.update(not_ortalamasi=F("not_ortalamasi") + 5)
```

NOT MEASURED Needs a Django project and a database server. The gain is in query count: n UPDATEs → 1.

Source: Django belgeleri: bulk_update · update() · docs.djangoproject.com/en/stable/ref/models/querysets/#bulk-update

---

YK035 medium Gives advice not measured

len() on a query result just to count

len(queryset) loads every object. If you only need the number, use .count(); the database runs COUNT(*).

WHY DOES IT WASTE ENERGY?

len() fetches the whole result and builds a model object for every row; count() asks for a single number.

WHEN IS IT FINE?

If the objects will be used afterwards anyway, len() does not add a query.

WORK DONE **n objects → 1 number**

WASTEFUL

```
def aktif_kullanici_sayisi():
    return len(Kullanici.objects.filter(aktif=True))
```

EFFICIENT

```
def aktif_kullanici_sayisi():
    return Kullanici.objects.filter(aktif=True).count()
```

NOT MEASURED Needs a Django project and a database server. The gain is in data moved: n objects → 1 number.

Source: Django belgeleri: count() ve len() · docs.djangoproject.com/en/stable/ref/models/querysets/#count

---

YK036 info Gives advice — no difference

Query for one row without LIMIT

A query with no WHERE and no LIMIT prepares to return the whole table while fetchone() uses only the first row. Add LIMIT 1.

WHY DOES IT WASTE ENERGY?
Without WHERE and LIMIT the database prepares the whole result; LIMIT 1 lets it stop at the first row.

WHEN IS IT FINE?
No difference if the table has a single row.

WORK DONE `whole result → first row`

WASTEFUL

```
def ilk_ayar(db):
    return db.execute("SELECT anahtar, deger FROM
ayar").fetchone()
```

EFFICIENT

```
def ilk_ayar(db):
    return db.execute("SELECT anahtar, deger FROM ayar LIMIT
1").fetchone()
```

MEASURED time 1.4 µs → 1.3 µs · memory 718 B → 718 B · same result ✓

An SQLite cursor already stops at the first row, so the difference is negligible. It shows on server databases that prepare the result and send it over the network.

Source: SQL: LIMIT ile erken durma · www.sqlite.org/lang_select.html#limitoffset

---

YK037 medium Gives advice — 1.57× faster

Random row with ORDER BY RANDOM()

ORDER BY RANDOM()/RAND() reads and sorts the whole table. Pick a random id and fetch that row, or use TABLESAMPLE.

WHY DOES IT WASTE ENERGY?
Every row gets a random number and the whole table is sorted; one row costs work proportional to the table size.

WHEN IS IT FINE?
Fine for very small tables.

WORK DONE `O(n log n) → O(1)`

WASTEFUL

```
def rastgele_soru(db):
    return db.execute("SELECT * FROM soru ORDER BY RANDOM()
LIMIT 1").fetchone()
```

EFFICIENT

```
import random


def rastgele_soru(db):
    en_kucuk, en_buyuk = db.execute("SELECT MIN(id), MAX(id)
FROM soru").fetchone()
    secilen = random.randint(en_kucuk, en_buyuk)
    return db.execute("SELECT id, metin FROM soru WHERE id >= ?
ORDER BY id LIMIT 1", (secilen,)).fetchone()
```

MEASURED time 1.1 ms → 721 µs · memory 838 B → 1.1 KB · random output

The result is random, so outputs were not compared; only time was measured.

Source: PostgreSQL belgeleri: TABLESAMPLE · www.postgresql.org/docs/current/sql-select.html#SQL-FROM

YK038 info Gives advice — 2.87× faster

LIKE search with a leading wildcard

LIKE '%...' cannot use an index and scans the whole table. Use a full-text index for text search.

WHY DOES IT WASTE ENERGY?
A B-tree index can do prefix searches; a pattern starting with '%' cannot use it and scans every row.

WHEN IS IT FINE?
Acceptable for small tables or rare queries.

WORK DONE `O(log n) → O(n)`

WASTEFUL

```
def ada_gore_ara(db, parca):
    return db.execute("SELECT id, ad FROM musteri WHERE ad LIKE
'%' || ? || '%'", (parca,)).fetchall()
```

EFFICIENT

```
def ada_gore_ara(db, parca):
    # musteri_ara: CREATE VIRTUAL TABLE musteri_ara USING
fts5(ad, tokenize='trigram')
    ifade = '"' + parca.replace('"', '""') + '"'
    return db.execute("SELECT rowid, ad FROM musteri_ara WHERE
musteri_ara MATCH ? ORDER BY rowid", (ifade,)).fetchall()
```

MEASURED time 3.0 ms → 1.1 ms · memory 115.0 KB → 115.1 KB · same result ✓

The good version uses an FTS5 trigram index; it does the same substring search and returns the same rows. Building the index is not part of the measurement.

Source: Veritabanı dizinleri: B-tree önek araması · www.sqlite.org/optoverview.html#the_like_optimization

---

YK039 medium Gives advice **8.89× faster**

Aggregating in Python

Fetching every row to compute sum/min/max in Python moves needless data. Let the database do it with SUM(), MIN(), MAX().

WHY DOES IT WASTE ENERGY?

Instead of moving every row to sum it, SUM() computes the result inside the database and returns one value.

WHEN IS IT FINE?

No extra cost if the rows are fetched for other work anyway.

WORK DONE **n rows → 1 value**

WASTEFUL

```
def toplam_ciro(db):
    return sum(satir[0] for satir in db.execute("SELECT tutar
FROM siparis"))
```

EFFICIENT

```
def toplam_ciro(db):
    return db.execute("SELECT SUM(tutar) FROM
siparis").fetchone()[0]
```

MEASURED time 5.5 ms → 617 µs · memory 1,014 B → 675 B · same result ✓

Source: Django belgeleri: aggregate() · `docs.djangoproject.com/en/stable/topics/db/aggregation/`

YK040 medium Gives advice **1.89× faster**

Formatted SQL query

A query built with an f-string or format is a different text each time, so the database cannot reuse its plan, and it is open to SQL injection. Use parameters.

WHY DOES IT WASTE ENERGY?

Every distinct query text is parsed and planned again; with parameters the text stays the same so the prepared plan is reused. It also prevents SQL injection.

WHEN IS IT FINE?

Formatting may be needed for parts that cannot be parameters, such as table names; those must come from a trusted source.

WORK DONE **n plans → 1 plan**

WASTEFUL

```
def musteri_bul(db, sehir):
    return db.execute(f"SELECT id, ad FROM musteri WHERE sehir
= '{sehir}'").fetchall()
```

EFFICIENT

```
def musteri_bul(db, sehir):
    return db.execute("SELECT id, ad FROM musteri WHERE sehir =
?", (sehir,)).fetchall()
```

MEASURED time 3.1 ms → 1.6 ms · memory 274.8 KB → 231.1 KB · same result ✓

Measured with 300 different values. Python's sqlite3 module caches the last 128 query texts; when each value is a separate text the cache runs out and the query is recompiled every time.

Source: PEP 249 (DB-API): parametre işaretleri · `peps.python.org/pep-0249/#paramstyle`

---

02 Loops and algorithms

18 rules

YK002 high Engine fixes it — 375× faster

List lookup inside a loop

An “in” test on a list scans it from start to end every time. Convert it to a set() before the loop; lookups become constant-time.

WHY DOES IT WASTE ENERGY?
An 'in' test on a list compares elements one by one (O(n)); inside a loop that is n × m comparisons. A set uses a hash table and answers in constant time.

WHEN IS IT FINE?
If the list has only a few elements, or you search it only once, converting to a set is not worth it.

WORK DONE `O(n·m) → O(n + m)`

WASTEFUL

```
def yoklama(gelenler, kayit):
    kayitlilar = list(kayit)
    return [no for no in gelenler if no in kayitlilar]
```

EFFICIENT

```
def yoklama(gelenler, kayit):
    kayitlilar = set(kayit)
    return [no for no in gelenler if no in kayitlilar]
```

MEASURED time 23.2 ms → 62 µs · memory 32.1 KB → 160.3 KB · same result ✓

The engine’s fix gave the same result too (321×).

Source: Python wiki: TimeComplexity · `wiki.python.org/moin/TimeComplexity`

YK003 medium Engine fixes it — 109× faster

Repeating loop-invariant work

This call returns the same value on every iteration. Do it once before the loop and keep the result.

WHY DOES IT WASTE ENERGY?
A loop-invariant call (JSON parsing, regex compiling) produces the same result again on every iteration, multiplying the work by the loop length.

WHEN IS IT FINE?
If the call has side effects or must return a fresh result each time (e.g. the file changes), do not hoist it.

WORK DONE `n units of work → 1`

WASTEFUL

```
import json


def fiyatla(urunler, ayar_metni):
    sonuc = []
    for u in urunler:
        ayar = json.loads(ayar_metni)
        sonuc.append(u["fiyat"] * (1 + ayar["kdv"]))
    return sonuc
```

EFFICIENT

```
import json


def fiyatla(urunler, ayar_metni):
    kdv = json.loads(ayar_metni)["kdv"]
    return [u["fiyat"] * (1 + kdv) for u in urunler]
```

MEASURED time 18.1 ms → 166 µs · memory 101.8 KB → 93.6 KB · same result ✓

The engine’s fix gave the same result too (83.9×).

Source: Python wiki: PerformanceTips · `wiki.python.org/moin/PythonSpeed/PerformanceTips`

YK004 high Engine fixes it **1,681× faster**

Uncached double recursion

The function calls itself more than once, recomputing the same sub-results exponentially. Add @functools.lru_cache or rewrite it as a loop.

WHY DOES IT WASTE ENERGY?

Recursion like f(n-1) + f(n-2) solves the same subproblems again and again; the number of calls grows exponentially. A cache computes each sub-result once.

WHEN IS IT FINE?

If the function has side effects (printing, drawing) or its arguments are unhashable (lists, dicts), it cannot be cached.

WORK DONE $O(2^n) \rightarrow O(n)$

WASTEFUL

```
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)
```

EFFICIENT

```
from functools import lru_cache


@lru_cache(maxsize=None)
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)
```

MEASURED time 5.5 ms → 3.3 µs · memory 224 B → 1.9 KB · same result ✓

The cache was cleared before every call; the good version also computed from scratch each time.

The engine's fix gave the same result too (1,683×).

Source: Python belgeleri: functools.lru_cache · docs.python.org/3/library/functools.html#functools.lru_cache

---

YK006 info Gives advice **158× faster**

Nested loop ($O(n^2)$)

Nested loops slow down quadratically as data grows. If the inner loop is matching, index the data in a dict and do one pass.

WHY DOES IT WASTE ENERGY?

Two nested loops take n × m steps; ten times the data means a hundred times the work. An inner loop that does matching can become one pass with a dict.

WHEN IS IT FINE?

If every pair genuinely has to be compared (e.g. a distance matrix), nested loops are natural.

WORK DONE $O(n \cdot m) \rightarrow O(n + m)$

WASTEFUL

```
def eslestir(ogrenciler, dersler):
    eslesme = []
    for o in ogrenciler:
        for d in dersler:
            if d["ogrenci_id"] == o["id"]:
                eslesme.append((o["ad"], d["ad"]))
    return eslesme
```

EFFICIENT

```
from collections import defaultdict


def eslestir(ogrenciler, dersler):
    ogrencinin = defaultdict(list)
    for d in dersler:
        ogrencinin[d["ogrenci_id"]].append(d["ad"])
    return [(o["ad"], ders) for o in ogrenciler for ders in
  ogrencinin.get(o["id"], ())]
```

MEASURED time 40.9 ms → 259 µs · memory 14.1 KB → 86.9 KB · same result ✓

Source: Python wiki: TimeComplexity · wiki.python.org/moin/TimeComplexity

---

YK011 medium Engine fixes it **4.26× faster**

Sorting to find the smallest or largest

Sorting the whole list to take its first or last element is O(n log n). min() and max() find the same result in one pass (O(n)).

WHY DOES IT WASTE ENERGY?

Sorting is O(n log n) work plus a full copy; finding the smallest or largest needs only one pass (O(n)).

WHEN IS IT FINE?

If the whole sorted list is used elsewhere too, the sort is not wasted.

WORK DONE $O(n \log n) \rightarrow O(n)$

WASTEFUL

```
def en_ucuz_urun(urunler):
    return sorted(urunler, key=lambda u: u["fiyat"])[0]
```

EFFICIENT

```
def en_ucuz_urun(urunler):
    return min(urunler, key=lambda u: u["fiyat"])
```

MEASURED time 3.5 ms → 827 µs · memory 468.9 KB → 208 B · same result ✓

The engine's fix gave the same result too (4.38×).

Source: Ruff FURB192 sorted-min-max · docs.astral.sh/ruff/rules/sorted-min-max/

YK013 high Gives advice **64.3× faster**

index() or count() on a list in a loop

index() and count() scan the whole list, so the work grows quadratically in a loop. Use a dict for positions and collections.Counter for counts.

WHY DOES IT WASTE ENERGY?

list.index and list.count scan the list on every call; in a loop that is n × m steps. With a dict or Counter built once, lookups are constant-time.

WHEN IS IT FINE?

Fine for small lists or a single call.

WORK DONE `O(n·m) → O(n + m)`

WASTEFUL

```
def siralama_puani(ogrenciler, sinif_listesi):
    sira = list(sinif_listesi)
    return [sira.index(o) for o in ogrenciler]
```

EFFICIENT

```
def siralama_puani(ogrenciler, sinif_listesi):
    sira = {}
    for i, ad in enumerate(sinif_listesi):
        sira.setdefault(ad, i)
    return [sira[o] for o in ogrenciler]
```

MEASURED time 14.1 ms → 220 µs · memory 57.0 KB → 219.8 KB · same result ✓

Source: Python wiki: TimeComplexity · `wiki.python.org/moin/TimeComplexity`

YK026 high Gives advice **292× faster**

Searching dict values in a loop

x in d.values() scans every value one by one. Build a set of the values or keep a reverse dict.

WHY DOES IT WASTE ENERGY?

The values() view is not a hash table; 'in' compares every value one by one.

WHEN IS IT FINE?

Fine for small dicts or a single lookup.

WORK DONE `O(n·m) → O(n + m)`

WASTEFUL

```
def kayitli_eposta(adaylar, kullanicilar):
    return [e for e in adaylar if e in kullanicilar.values()]
```

EFFICIENT

```
def kayitli_eposta(adaylar, kullanicilar):
    epostalar = set(kullanicilar.values())
    return [e for e in adaylar if e in epostalar]
```

MEASURED time 16.4 ms → 56 µs · memory 4.2 KB → 160.3 KB · same result ✓

Source: Python wiki: TimeComplexity · `wiki.python.org/moin/TimeComplexity`

YK028 medium Gives advice **4.70× faster**

Re-processing unchanging text in a loop

A string that never changes during the loop is split or converted again on every step. Do it once before the loop.

WHY DOES IT WASTE ENERGY?

Splitting or lower-casing an unchanging string on every step produces the same result again and again.

WHEN IS IT FINE?

If the string changes in the loop, the operation is needed every step.

WORK DONE `n units of work → 1`

WASTEFUL

```
def yasakli_kelime_var_mi(mesajlar, yasakli_metin):
    sonuc = []
    for m in mesajlar:
        yasaklar = yasakli_metin.lower().split(",")
        sonuc.append(any(y in m for y in yasaklar))
    return sonuc
```

EFFICIENT

```
def yasakli_kelime_var_mi(mesajlar, yasakli_metin):
    yasaklar = yasakli_metin.lower().split(",")
    return [any(y in m for y in yasaklar) for m in mesajlar]
```

MEASURED time 4.9 ms → 1.0 ms · memory 30.2 KB → 28.1 KB · same result ✓

Source: Python wiki: PerformanceTips · `wiki.python.org/moin/PythonSpeed/PerformanceTips`

YK041 medium Gives advice **20.1× faster**

Building a whole list to take the first match

[...][0] builds the entire list and uses only the first element. next((x for x in ... if ...), None) stops at the first match.

WHY DOES IT WASTE ENERGY?

The comprehension produces every element; if only the first is needed, next() stops at the first match.

WHEN IS IT FINE?

No difference if the list is used for other things too.

WORK DONE **O(n) → up to the first match**

WASTEFUL

```
def ilk_buyuk_siparis(siparisler):
    return [s for s in siparisler if s["tutar"] > 1000][0]
```

EFFICIENT

```
def ilk_buyuk_siparis(siparisler):
    return next((s for s in siparisler if s["tutar"] > 1000),
  None)
```

MEASURED time 398 µs → 20 µs · memory 80 B → 408 B · same result ✓

Source: Python belgeleri: next() ve üreteçler · docs.python.org/3/library/functions.html#next

---

YK042 medium Gives advice **2,438× faster**

Copying everything to take the first element

list(x)[0] copies every element. next(iter(x)) takes only the first one.

WHY DOES IT WASTE ENERGY?

list(x) copies every element; next(iter(x)) takes only the first.

WHEN IS IT FINE?

No difference if the list is used for other things too.

WORK DONE **O(n) → O(1)**

WASTEFUL

```
def ilk_anahtar(sozluk):
    return list(sozluk.keys())[0]
```

EFFICIENT

```
def ilk_anahtar(sozluk):
    return next(iter(sozluk))
```

MEASURED time 230 µs → 0.1 µs · memory 390.7 KB → 72 B · same result ✓

Source: Python belgeleri: next(iter(...)) · docs.python.org/3/library/functions.html#next

---

YK043 info Gives advice **13.7× faster**

Full sort to take the top k

sorted(x)[:k] sorts the whole list. For small k, heapq.nsmallest(k, x) or heapq.nlargest(k, x) does less work.

WHY DOES IT WASTE ENERGY?

A full sort is O(n log n); heapq.nsmallest needs only O(n log k) for small k and no full copy.

WHEN IS IT FINE?

When k is close to n, a full sort can be faster.

WORK DONE **O(n log n) → O(n log k)**

WASTEFUL

```
def en_iyi_uc(puanlar):
    return sorted(puanlar, reverse=True)[:3]
```

EFFICIENT

```
import heapq


def en_iyi_uc(puanlar):
    return heapq.nlargest(3, puanlar)
```

MEASURED time 7.4 ms → 542 µs · memory 585.9 KB → 316 B · same result ✓

Source: Python belgeleri: heapq.nsmallest · docs.python.org/3/library/heapq.html#heapq.nsmallest

---

YK044 high Gives advice

439× faster

Building a collection for a lookup inside a loop

x in set(...) or list(...) rebuilds the collection on every iteration. Build the set once before the loop.

WHY DOES IT WASTE ENERGY?

set(...) refills a hash table from the whole source on every step; the expected speed-up is lost and the build cost is added.

WHEN IS IT FINE?

If the source changes every step, rebuilding is necessary.

WORK DONE O(n·m) → O(n + m)

WASTEFUL

```
def kayitli_olanlar(gelenler, kayitlar):
    return [g for g in gelenler if g in set(kayitlar)]
```

EFFICIENT

```
def kayitli_olanlar(gelenler, kayitlar):
    kayitli = set(kayitlar)
    return [g for g in gelenler if g in kayitli]
```

MEASURED time 16.0 ms → 36 µs · memory 164.4 KB → 160.3 KB · same result ✓

Source: Python wiki: TimeComplexity · wiki.python.org/moin/TimeComplexity

---

YK045 medium Gives advice

462× faster

list.remove() inside a loop

remove() scans the list and shifts the rest; in a loop the work grows quadratically. Build a new list of the items to keep, or use a set.

WHY DOES IT WASTE ENERGY?

remove first searches for the element and then shifts the rest; in a loop the work becomes quadratic.

WHEN IS IT FINE?

Fine when removing only a few elements.

WORK DONE O(n·m) → O(n + m)

WASTEFUL

```
def gecersizleri_cikar(kayitlar, gecersizler):
    kalan = list(kayitlar)
    for g in gecersizler:
        kalan.remove(g)
    return kalan
```

EFFICIENT

```
def gecersizleri_cikar(kayitlar, gecersizler):
    cikarilacak = set(gecersizler)
    return [k for k in kayitlar if k not in cikarilacak]
```

MEASURED time 170.2 ms → 368 µs · memory 156.4 KB → 278.4 KB · same result ✓

The records are unique; with duplicates remove deletes only the first, the good version all of them.

Source: Python wiki: TimeComplexity · wiki.python.org/moin/TimeComplexity

---

YK047 medium Gives advice

8.20× faster

Joining a growing list on every iteration

Joining a list that grows every step from scratch is quadratic work. Join once after the loop.

WHY DOES IT WASTE ENERGY?

join rebuilds the whole string from the list on every call; as the list grows each step costs more and the total becomes quadratic.

WHEN IS IT FINE?

If every intermediate result is truly needed, grow the last string step by step instead.

WORK DONE O(n²) → O(n)

WASTEFUL

```
def ilerleme_mesajlari(adimlar):
    parcalar = []
    mesajlar = []
    for a in adimlar:
        parcalar.append(a)
        mesajlar.append(" > ".join(parcalar))
    return mesajlar
```

EFFICIENT

```
from itertools import accumulate


def ilerleme_mesajlari(adimlar):
    return list(accumulate(adimlar, lambda once, a: once + " >
  " + a))
```

MEASURED time 3.9 ms → 482 µs · memory 6.6 MB → 6.6 MB · same result ✓

Source: Python wiki: PerformanceTips (str.join) · wiki.python.org/moin/PythonSpeed/PerformanceTips

---

YK048 medium Gives advice **114× faster**

Deep copy inside a loop

copy.deepcopy walks and copies the whole object graph; inside a loop it is very expensive. Copy only what changes or use immutable structures.

WHY DOES IT WASTE ENERGY?

deepcopy walks the whole object graph, copies every sub-object and keeps a memo to track cycles; it is one of the most expensive operations in Python.

WHEN IS IT FINE?

May be unavoidable when truly independent mutable copies are needed; often copying only the changed field is enough.

WORK DONE **full graph copy → only the changed part**

WASTEFUL

```
import copy


def raporlar(sablon, ogrenciler):
    sonuc = []
    for o in ogrenciler:
        r = copy.deepcopy(sablon)
        r["ad"] = o
        sonuc.append(r)
    return sonuc
```

EFFICIENT

```
def raporlar(sablon, ogrenciler):
    return [{**sablon, "ad": o} for o in ogrenciler]
```

MEASURED time 12.4 ms → 109 µs · memory 1.7 MB → 184.0 KB · same result ✓

The good version makes a shallow copy: the nested 'dersler' list is shared between reports. If reports will be changed one by one later, copy only the part that changes.

Source: Python belgeleri: copy · docs.python.org/3/library/copy.html

---

YK049 info Gives advice **1.15× slower**

try/except inside a loop

Before Python 3.11, setting up a try block on every iteration adds overhead. Move the try/except outside the loop where possible.

WHY DOES IT WASTE ENERGY?

Before Python 3.11, entering a try block did setup work for exception handling. With 3.11's zero-cost exceptions this cost largely disappeared.

WHEN IS IT FINE?

On Python 3.11+, the cost is negligible unless an exception is raised.

WORK DONE **setup every step → none (3.11+)**

WASTEFUL

```
def sayilara_cevir(metinler):
    sonuc = []
    for m in metinler:
        try:
            sonuc.append(int(m))
        except ValueError:
            sonuc.append(0)
    return sonuc
```

EFFICIENT

```
def _sayi(metin):
    try:
        return int(metin)
    except ValueError:
        return 0


def sayilara_cevir(metinler):
    return [_sayi(m) for m in metinler]
```

MEASURED time 2.3 ms → 2.6 ms · memory 655.1 KB → 655.0 KB · same result ✓

Entering a try block has cost nothing since Python 3.11; no difference is expected on the version used here.

Source: Ruff PERF203 try-except-in-loop · docs.astral.sh/ruff/rules/try-except-in-loop/

YK059 `info` `Gives advice` **2.71× faster**

import inside a loop

An import inside a loop looks the module up again on every step. Move it to the top of the file.

WHY DOES IT WASTE ENERGY?

import looks up sys.modules and binds the name every time; even when cached it adds work per step.

WHEN IS IT FINE?

Lazy-importing a heavy optional module inside a function makes sense, but outside the loop.

WORK DONE `n lookups → 1`

WASTEFUL

```
def tarihleri_isle(metinler):
    sonuc = []
    for m in metinler:
        from datetime import datetime
        sonuc.append(datetime.fromisoformat(m))
    return sonuc
```

EFFICIENT

```
from datetime import datetime


def tarihleri_isle(metinler):
    return [datetime.fromisoformat(m) for m in metinler]
```

MEASURED time 7.1 ms → 2.6 ms · memory 950.3 KB → 950.3 KB · same result ✓

Source: Python wiki: PerformanceTips (Import Statement Overhead) · `wiki.python.org/moin/PythonSpeed/PerformanceTips#Import_Statement_Overhead`

YK068 `medium` `Gives advice` **2.83× faster**

Loop-invariant maths in a loop

This math call gives the same result on every iteration. Compute it once before the loop.

WHY DOES IT WASTE ENERGY?

An unchanging maths calculation is repeated every step; the result can be computed once and kept.

WHEN IS IT FINE?

No difference for trivially cheap values (constants like math.pi).

WORK DONE `n calculations → 1`

WASTEFUL

```
import math


def ucgen_yukseklikleri(kenarlar):
    return [k * math.sqrt(3) / 2 for k in kenarlar]
```

EFFICIENT

```
import math


def ucgen_yukseklikleri(kenarlar):
    oran = math.sqrt(3) / 2
    return [k * oran for k in kenarlar]
```

MEASURED time 3.0 ms → 1.1 ms · memory 1.6 MB → 1.6 MB · same result ✓

Source: Python wiki: PerformanceTips · `wiki.python.org/moin/PythonSpeed/PerformanceTips`

---

03 Data structures and Python idioms

31 rules

YK009 medium Gives advice

71.8× faster

Adding or removing at the front of a list

pop(0) and insert(0, ...) shift every element of the list. Use collections.deque: popleft and appendleft run in constant time.

WHY DOES IT WASTE ENERGY?
A list is a contiguous array; removing or inserting at the front shifts every other element. A deque works in constant time at both ends.

WHEN IS IT FINE?
With very short lists the difference is too small to measure.

WORK DONE O(n²) → O(n)

WASTEFUL

```
def sirayla_isle(isler):
    biten = []
    while isler:
        biten.append(isler.pop(0))
    return biten
```

EFFICIENT

```
from collections import deque


def sirayla_isle(isler):
    kuyruk = deque(isler)
    isler.clear()
    biten = []
    while kuyruk:
        biten.append(kuyruk.popleft())
    return biten
```

MEASURED time 91.2 ms → 1.3 ms · memory 300.8 KB → 270.8 KB · same result ✓

Source: Python belgeleri: collections.deque · docs.python.org/3/library/collections.html#collections.deque

YK016 medium Gives advice

13.9× faster

A list inside any() or all()

The square-bracket list computes every element first, so any() and all() cannot stop early. Drop the brackets and use a generator.

WHY DOES IT WASTE ENERGY?
any([...]) computes the whole list first and loses the chance to stop at the first True. With a generator, any stops at the first match.

WHEN IS IT FINE?
If every element must be evaluated for its side effects (deliberately), keep the list.

WORK DONE n → up to the first match

WASTEFUL

```
def gecersiz_var_mi(kayitlar):
    return any([dogrula(k) is False for k in kayitlar])
```

EFFICIENT

```
def gecersiz_var_mi(kayitlar):
    return any(dogrula(k) is False for k in kayitlar)
```

MEASURED time 624 µs → 45 µs · memory 169.0 KB → 416 B · same result ✓

The invalid record is near the start of the list (position 1,000). Had it been at the end, both versions would do the same work.

Source: Ruff C419 unnecessary-comprehension-in-call · docs.astral.sh/ruff/rules/unnecessary-comprehension-in-call/

YK017 medium Gives advice

118× faster

Rebuilding a list in a loop

L = L + [x] copies the whole list every step; the work grows quadratically. Use L.append(x) or L.extend(...).

WHY DOES IT WASTE ENERGY?
The + operator copies both lists into a new list every time; append adds to the end in place.

WHEN IS IT FINE?
If another variable refers to the old list and that version must be preserved, the copy may be intentional.

WORK DONE O(n²) → O(n)

WASTEFUL

```
def ciftleri_topla(sayilar):
    sonuc = []
    for s in sayilar:
        if s % 2 == 0:
            sonuc = sonuc + [s]
    return sonuc
```

EFFICIENT

```
def ciftleri_topla(sayilar):
    return [s for s in sayilar if s % 2 == 0]
```

MEASURED time 37.7 ms → 320 µs · memory 78.2 KB → 40.9 KB · same result ✓

Source: Python wiki: TimeComplexity · wiki.python.org/moin/TimeComplexity

YK024 medium Gives advice **111× faster**

Copying through JSON

json.loads(json.dumps(x)) turns data into text and parses it back, turning tuples into lists and int keys into strings. Copy only what changes: {**x, 'field': new}. If a fully independent copy is needed, copy.deepcopy keeps the types.

WHY DOES IT WASTE ENERGY?

A JSON round trip turns every value into text and parses it back, silently turning tuples into lists and int keys into strings. Usually the whole structure does not need copying either: a new dict holding only the changed field gives the same result while sharing the rest.

WHEN IS IT FINE?

If the copy's nested fields will be modified later, a shallow copy is not enough; use copy.deepcopy. It keeps types but is even slower than JSON, so use it only when truly needed.

WORK DONE `full copy → only the changed field`

WASTEFUL

```
import json


def kullanici_ayari(varsayilan, tema):
    ayar = json.loads(json.dumps(varsayilan))
    ayar["tema"] = tema
    return ayar
```

EFFICIENT

```
def kullanici_ayari(varsayilan, tema):
    return {**varsayilan, "tema": tema}
```

MEASURED time 16 µs → 0.1 µs · memory 4.7 KB → 120 B · same result ✓

The good version copies only the changed field; nested structures are shared with the defaults. The comparison also checked that the defaults stayed unchanged.

Source: Python belgeleri: copy · json (dönüşüm tablosu) · `docs.python.org/3/library/json.html#py-to-json-table`

YK025 medium Gives advice **178× faster**

Joining lists with sum()

sum(lists, []) builds a new list at every step; the work grows quadratically. Use itertools.chain.from_iterable.

WHY DOES IT WASTE ENERGY?

sum builds a brand-new list at every step from the accumulated list and the next one; total copying grows quadratically. chain yields elements in order without copying.

WHEN IS IT FINE?

With a few short lists the difference is negligible.

WORK DONE `O(n²) → O(n)`

WASTEFUL

```
def hepsini_birlestir(gruplar):
    return sum(gruplar, [])
```

EFFICIENT

```
from itertools import chain


def hepsini_birlestir(gruplar):
    return list(chain.from_iterable(gruplar))
```

MEASURED time 13.9 ms → 78 µs · memory 156.2 KB → 83.3 KB · same result ✓

Source: Python belgeleri: sum() (itertools.chain önerisi) · `docs.python.org/3/library/functions.html#sum`

YK029 info Engine fixes it **1.20× faster**

Needless list copy

Copying with list() before looping costs memory and time. Objects such as range, map and zip can be iterated directly.

WHY DOES IT WASTE ENERGY?

list() copies every element into memory first; range, map and zip are lazy and can be iterated directly.

WHEN IS IT FINE?

If the source is modified while iterating (deleting from a dict), the copy is genuinely needed.

WORK DONE `O(n) memory → O(1)`

WASTEFUL

```
def kareler(n):
    toplam = 0
    for i in list(range(n)):
        toplam += i * i
    return toplam
```

EFFICIENT

```
def kareler(n):
    toplam = 0
    for i in range(n):
        toplam += i * i
    return toplam
```

MEASURED time 9.8 ms → 8.2 ms · memory 7.6 MB → 176 B · same result ✓

Similar time; the real gain is memory: range produces numbers one at a time, list() holds them all.

The engine's fix gave the same result too (1.24×).

Source: Ruff PERF101 unnecessary-list-cast · docs.astral.sh/ruff/rules/unnecessary-list-cast/

YK046 medium Gives advice **139× faster**

del L[0] inside a loop

Deleting the first element shifts every other element. Use collections.deque and popleft().

WHY DOES IT WASTE ENERGY?

Deleting from the front of a list shifts every other element; deque.popleft is constant time.

WHEN IS IT FINE?

Small difference for very short lists.

WORK DONE `O(n²) → O(n)`

WASTEFUL

```
def kuyrugu_isle(isler):
    sonuc = []
    while isler:
        sonuc.append(isler[0] * 2)
        del isler[0]
    return sonuc
```

EFFICIENT

```
from collections import deque


def kuyrugu_isle(isler):
    kuyruk = deque(isler)
    isler.clear()
    sonuc = []
    while kuyruk:
        sonuc.append(kuyruk.popleft() * 2)
    return sonuc
```

MEASURED time 220.2 ms → 1.6 ms · memory 1.1 MB → 1.2 MB · same result ✓

Source: Python belgeleri: collections.deque · docs.python.org/3/library/collections.html#collections.deque

---

YK050 info Gives advice **1.15× faster**

Manual list comprehension

Appending to an empty list in a loop is slower than the equivalent list comprehension.

WHY DOES IT WASTE ENERGY?

Each append in a loop is an attribute lookup and a method call; a comprehension uses a dedicated bytecode (LIST_APPEND).

WHEN IS IT FINE?

With a complex body an explicit loop may read better; the difference is usually 10-30%.

WORK DONE `n method calls → 0`

WASTEFUL

```
def kareler(sayilar):
    sonuc = []
    for s in sayilar:
        sonuc.append(s * s)
    return sonuc
```

EFFICIENT

```
def kareler(sayilar):
    return [s * s for s in sayilar]
```

MEASURED time 520 µs → 452 µs · memory 793.4 KB → 793.4 KB · same result ✓

Source: Ruff PERF401 manual-list-comprehension · docs.astral.sh/ruff/rules/manual-list-comprehension/

YK051 info Gives advice

4.29× faster

Manual list copy

Instead of appending elements one by one in a loop, use list(x) or L.extend(x).

WHY DOES IT WASTE ENERGY?

Copying element by element runs at Python level; list(x) and extend copy in C in one go.

WHEN IS IT FINE?

If elements are transformed while copying, use a comprehension.

WORK DONE `Python loop → C copy`

WASTEFUL

```
def kopyala(kaynak):
    hedef = []
    for x in kaynak:
        hedef.append(x)
    return hedef
```

EFFICIENT

```
def kopyala(kaynak):
    return list(kaynak)
```

MEASURED time 262 µs → 61 µs · memory 169.0 KB → 156.3 KB · same result ✓

Source: Ruff PERF402 manual-list-copy · docs.astral.sh/ruff/rules/manual-list-copy/

YK052 info Gives advice

no difference

Manual dict comprehension

Filling an empty dict in a loop is slower than the equivalent dict comprehension.

WHY DOES IT WASTE ENERGY?

Each assignment in the loop is a separate Python step; a dict comprehension does the same with a dedicated bytecode.

WHEN IS IT FINE?

With a complex body an explicit loop can be clearer.

WORK DONE `n steps → comprehension`

WASTEFUL

```
def numara_sozlugu(ogrenciler):
    sozluk = {}
    for o in ogrenciler:
        sozluk[o["no"]] = o["ad"]
    return sozluk
```

EFFICIENT

```
def numara_sozlugu(ogrenciler):
    return {o["no"]: o["ad"] for o in ogrenciler}
```

MEASURED time 797 µs → 787 µs · memory 864.1 KB → 864.1 KB · same result ✓

Source: Ruff PERF403 manual-dict-comprehension · docs.astral.sh/ruff/rules/manual-dict-comprehension/

YK053 info Gives advice

1.76× faster

Adding to a set one by one in a loop

Instead of s.add(x) in a loop, s.update(source) adds everything in one call.

WHY DOES IT WASTE ENERGY?

Each add is a method call; update adds every element in a C loop.

WHEN IS IT FINE?

If elements are transformed while adding, use a set comprehension.

WORK DONE `n calls → 1`

WASTEFUL

```
def etiketleri_topla(yazilar):
    etiketler = set()
    for e in yazilar:
        etiketler.add(e)
    return etiketler
```

EFFICIENT

```
def etiketleri_topla(yazilar):
    etiketler = set()
    etiketler.update(yazilar)
    return etiketler
```

MEASURED time 510 µs → 290 µs · memory 40.3 KB → 40.3 KB · same result ✓

Source: Ruff FURB142 for-loop-set-mutations · docs.astral.sh/ruff/rules/for-loop-set-mutations/

YK054 info Gives advice

4.90× faster

Unused key or value from items()

items() produces both key and value but one is unused. Iterate keys() or values() only.

WHY DOES IT WASTE ENERGY?

items() builds a key-value tuple every step; if one part is unused, keys() or values() does less work.

WHEN IS IT FINE?

The difference is small; it matters for large dicts.

WORK DONE `tuple → single value`

WASTEFUL

```
def toplam_puan(puanlar):
    toplam = 0
    for ad, puan in puanlar.items():
        toplam += puan
    return toplam
```

EFFICIENT

```
def toplam_puan(puanlar):
    return sum(puanlar.values())
```

MEASURED time 488 µs → 100 µs · memory 136 B → 112 B · same result ✓

Source: Ruff PERF102 incorrect-dict-iterator · `docs.astral.sh/ruff/rules/incorrect-dict-iterator/`

YK055 info Gives advice

1.32× faster

Iterating keys and looking each value up again

for k in d: ... d[k] does a second dict lookup every step. Use for k, v in d.items().

WHY DOES IT WASTE ENERGY?

d[k] hashes and searches the dict again every step; items() gives the value together with the key.

WHEN IS IT FINE?

If the dict is modified in the loop, a list of keys may be needed instead.

WORK DONE `2 lookups → 1`

WASTEFUL

```
def rapor(stok):
    satirlar = []
    for urun in stok:
        satirlar.append(f"{urun}: {stok[urun]}")
    return satirlar
```

EFFICIENT

```
def rapor(stok):
    return [f"{urun}: {adet}" for urun, adet in stok.items()]
```

MEASURED time 2.7 ms → 2.0 ms · memory 1.5 MB → 1.5 MB · same result ✓

Source: Pylint C0206 consider-using-dict-items · `pylint.readthedocs.io/en/stable/user_guide/messages/convention/consider-using-dict-items.html`

YK056 info Gives advice

1.20× faster

Indexing with range(len())

for i in range(len(x)): x[i] indexes on every step. Use for i, item in enumerate(x).

WHY DOES IT WASTE ENERGY?

x[i] indexes and bounds-checks every step; enumerate yields the element directly.

WHEN IS IT FINE?

If the index also accesses another sequence, combine zip and enumerate.

WORK DONE `indexing → element directly`

WASTEFUL

```
def numarali(ogrenciler):
    sonuc = []
    for i in range(len(ogrenciler)):
        sonuc.append((i + 1, ogrenciler[i]))
    return sonuc
```

EFFICIENT

```
def numarali(ogrenciler):
    return list(enumerate(ogrenciler, 1))
```

MEASURED time 1.8 ms → 1.5 ms · memory 1.7 MB → 1.7 MB · same result ✓

Source: Pylint C0200 consider-using-enumerate · `pylint.readthedocs.io/en/stable/user_guide/messages/convention/consider-using-enumerate.html`

YK057 info Gives advice 1.98× faster

Counting with a hand-written dict

if k in d: d[k] += 1 else: d[k] = 1 does two lookups every step. collections.Counter counts in one pass.

WHY DOES IT WASTE ENERGY?
Each step does a membership test, then a read and a write; Counter counts in C in one pass.

WHEN IS IT FINE?
If you merge values in a complex way rather than count, a manual dict may be needed.

WORK DONE `3 operations → 1`

WASTEFUL

```
def kelime_say(kelimeler):
    sayac = {}
    for k in kelimeler:
        if k in sayac:
            sayac[k] += 1
        else:
            sayac[k] = 1
    return sayac
```

EFFICIENT

```
from collections import Counter


def kelime_say(kelimeler):
    return Counter(kelimeler)
```

MEASURED time 2.5 ms → 1.2 ms · memory 38.1 KB → 38.2 KB · same result ✓

Source: Python belgeleri: collections.Counter · `docs.python.org/3/library/collections.html#collections.Counter`

YK058 info Gives advice 1.41× faster

Grouping with a hand-written dict

The if k not in d: d[k] = [] pattern adds a lookup every step. Use collections.defaultdict(list).

WHY DOES IT WASTE ENERGY?
Each step does an extra membership test; defaultdict creates the missing key itself on first access.

WHEN IS IT FINE?
If missing keys must not be created on read, a plain dict is safer.

WORK DONE `2 lookups → 1`

WASTEFUL

```
def sehre_gore(musteriler):
    gruplar = {}
    for m in musteriler:
        if m["sehir"] not in gruplar:
            gruplar[m["sehir"]] = []
        gruplar[m["sehir"]].append(m)
    return gruplar
```

EFFICIENT

```
from collections import defaultdict


def sehre_gore(musteriler):
    gruplar = defaultdict(list)
    for m in musteriler:
        gruplar[m["sehir"]].append(m)
    return dict(gruplar)
```

MEASURED time 1.2 ms → 820 µs · memory 175.4 KB → 176.8 KB · same result ✓

Source: Python belgeleri: collections.defaultdict · `docs.python.org/3/library/collections.html#collections.defaultdict`

YK060 info Gives advice 1.50× faster

dict()/list() call for an empty collection in a loop

dict() and list() need a name lookup and a call; {} and [] build the object directly.

WHY DOES IT WASTE ENERGY?
dict() is a global name lookup and a function call; {} is created directly by bytecode.

WHEN IS IT FINE?
dict(a=1) with keyword arguments may be preferred for readability.

WORK DONE `call → literal`

WASTEFUL

```
def bos_kayitlar(n):
    kayitlar = []
    for i in range(n):
        kayitlar.append(dict())
    return kayitlar
```

EFFICIENT

```
def bos_kayitlar(n):
    return [{} for _ in range(n)]
```

MEASURED time 1.4 ms → 919 µs · memory 1.4 MB → 1.4 MB · same result ✓

Source: Ruff C408 unnecessary-collection-call · `docs.astral.sh/ruff/rules/unnecessary-collection-call/`

YK061 info Gives advice — **2.12× faster**

map() or filter() with a lambda

map(lambda ...) makes a separate function call for every element. A comprehension or generator is faster.

WHY DOES IT WASTE ENERGY?

A lambda needs a Python function call (frame setup) per element; a comprehension runs the expression inline.

WHEN IS IT FINE?

map with a built-in (map(str, x)) is fast; the problem is the lambda.

WORK DONE **n function calls → 0**

WASTEFUL

```
def kdvli(fiyatlar):
    return list(map(lambda f: f * 1.2, fiyatlar))
```

EFFICIENT

```
def kdvli(fiyatlar):
    return [f * 1.2 for f in fiyatlar]
```

MEASURED time 1.0 ms → 490 µs · memory 637.9 KB → 637.7 KB · same result ✓

Source: Ruff C417 unnecessary-map · docs.astral.sh/ruff/rules/unnecessary-map/

YK062 info Engine fixes it — **no difference**

Needless double conversion

Calls such as list(sorted(x)), sorted(list(x)) and set(list(x)) copy the data one extra time for nothing.

WHY DOES IT WASTE ENERGY?

The inner call already produces a new list or set; the outer call copies the same data once more.

WHEN IS IT FINE?

If a type change is really needed, like tuple(sorted(x)), the double call is correct.

WORK DONE **2 copies → 1**

WASTEFUL

```
def benzersiz_sirali(etiketler):
    return list(sorted(set(etiketler)))
```

EFFICIENT

```
def benzersiz_sirali(etiketler):
    return sorted(set(etiketler))
```

MEASURED time 858 µs → 805 µs · memory 163.4 KB → 163.4 KB · same result ✓

Too small to measure: the extra copy runs in C and is negligible next to sorting. The gain is simplicity.

The engine's fix gave the same result too (1.26×).

Source: Ruff C414 unnecessary-double-cast-or-process · docs.astral.sh/ruff/rules/unnecessary-double-cast-or-process/

YK063 info Gives advice — **no difference**

reversed(sorted(...)) or sorted(...)[::-1]

Sorting and then reversing is an extra pass. Use sorted(x, reverse=True).

WHY DOES IT WASTE ENERGY?

Reversing after sorting is an extra pass and copy; reverse=True does it inside the sort.

WHEN IS IT FINE?

Mind ties: reversed(sorted()) reverses equal elements, while reverse=True keeps their order.

WORK DONE **2 passes → 1**

WASTEFUL

```
def buyukten_kucuge(puanlar):
    return list(reversed(sorted(puanlar)))
```

EFFICIENT

```
def buyukten_kucuge(puanlar):
    return sorted(puanlar, reverse=True)
```

MEASURED time 2.9 ms → 2.8 ms · memory 312.6 KB → 234.4 KB · same result ✓

Too small to measure: the reversed() copy is negligible next to sorting. reverse=True also keeps equal elements in order.

Source: Ruff C413 unnecessary-call-around-sorted · docs.astral.sh/ruff/rules/unnecessary-call-around-sorted/

YK064 info Engine fixes it **2.42× faster**

Comprehension that only copies

[x for x in y] only copies. list(y), set(y) or dict(y) do the same at C speed.

WHY DOES IT WASTE ENERGY?

The comprehension runs a Python loop per element; list(y) copies in C.

WHEN IS IT FINE?

If elements are transformed, the comprehension is right.

WORK DONE `Python loop → C copy`

WASTEFUL

```
def kopya(liste):
    return [x for x in liste]
```

EFFICIENT

```
def kopya(liste):
    return list(liste)
```

MEASURED time 411 µs → 170 µs · memory 434.0 KB → 390.7 KB · same result ✓

The engine's fix gave the same result too (2.39×).

Source: Ruff C416 unnecessary-comprehension · `docs.astral.sh/ruff/rules/unnecessary-comprehension/`

YK065 info Engine fixes it **1,089× less memory**

A list inside sum(), min() or max()

The square-bracket list holds every element in memory first. Use a generator: sum(x for x in y).

WHY DOES IT WASTE ENERGY?

The list holds every element in memory first; a generator yields them one by one with constant memory.

WHEN IS IT FINE?

For tiny inputs the list can be slightly faster; the gain is mainly memory.

WORK DONE `O(n) memory → O(1)`

WASTEFUL

```
def toplam_tutar(siparisler):
    return sum([s["tutar"] for s in siparisler])
```

EFFICIENT

```
def toplam_tutar(siparisler):
    return sum(s["tutar"] for s in siparisler)
```

MEASURED time 1.0 ms → 1.6 ms · memory 434.0 KB → 408 B · same result ✓

A generator takes no memory but can run a little slower than a list comprehension in CPython; the gain is memory.

The engine's fix gave the same result too (0.66×).

Source: Pylint R1728 consider-using-generator · `pylint.readthedocs.io/en/stable/user_guide/messages/refactor/consider-using-generator.html`

YK066 info Engine fixes it **1.80× faster**

Needless call around a literal

list([...]), set([...]) and tuple([...]) build a list and then copy it. Write [...], {...} or (...) directly.

WHY DOES IT WASTE ENERGY?

A temporary list is built and then copied; a literal creates the result directly.

WHEN IS IT FINE?

The difference is small, but it also simplifies the code.

WORK DONE `2 objects → 1`

WASTEFUL

```
def izinli_roller():
    return set(["yonetici", "editor", "okur"])
```

EFFICIENT

```
def izinli_roller():
    return {"yonetici", "editor", "okur"}
```

MEASURED time 137 µs → 76 µs · memory 219.7 KB → 219.6 KB · same result ✓

The engine's fix gave the same result too (1.84×).

Source: Ruff C405 / C406 / C409 / C410 / C411 · `docs.astral.sh/ruff/rules/unnecessary-literal-within-list-call/`

YK067 info Gives advice **216× less memory**

len(list(...)) just to count

len(list(x)) copies every element only to count them. Use len(x) for a sequence or sum(1 for _ in x) for an iterator.

WHY DOES IT WASTE ENERGY?
A copy of every element is built in memory just to count them.

WHEN IS IT FINE?
No difference if the list is used later anyway.

WORK DONE **O(n) memory → O(1)**

WASTEFUL

```
def satir_sayisi(dosya):
    return len(list(dosya))
```

EFFICIENT

```
def satir_sayisi(dosya):
    return sum(1 for _ in dosya)
```

MEASURED time 7.1 ms → 7.0 ms · memory 8.6 MB → 40.7 KB · same result ✓

Similar time; the real gain is memory: lines are not kept in a list while counting.

Source: Python belgeleri: len() · `docs.python.org/3/library/functions.html#len`

YK106 info Gives advice **1.21× faster**

Checking before removing from a set

if x in s: s.remove(x) looks the item up twice. s.discard(x) removes it in one lookup.

WHY DOES IT WASTE ENERGY?
in and remove each hash the value; discard removes it in one step.

WHEN IS IT FINE?
Use remove if a missing element should raise an error.

WORK DONE **2 lookups → 1**

WASTEFUL

```
def cikis_yap(oturumlar, kullanici):
    if kullanici in oturumlar:
        oturumlar.remove(kullanici)
```

EFFICIENT

```
def cikis_yap(oturumlar, kullanici):
    oturumlar.discard(kullanici)
```

MEASURED time 2.8 ms → 2.3 ms · memory 169.0 KB → 169.0 KB · same result ✓

Source: Ruff FURB132 check-and-remove-from-set · `docs.astral.sh/ruff/rules/check-and-remove-from-set/`

YK107 info Gives advice **11.0× faster**

Copying to reverse a list

L = L[::-1] creates a new list. Use L.reverse() to reverse in place.

WHY DOES IT WASTE ENERGY?
[::-1] allocates a new list; reverse() reverses in place.

WHEN IS IT FINE?
A copy is needed if the original list must not change.

WORK DONE **copy → in place**

WASTEFUL

```
def ters_cevir(adimlar):
    adimlar = adimlar[::-1]
    return adimlar
```

EFFICIENT

```
def ters_cevir(adimlar):
    adimlar.reverse()
    return adimlar
```

MEASURED time 910 µs → 83 µs · memory 781.2 KB → 0 B · same result ✓

The good version reverses the list in place, so the caller's list changes too. If the original order is needed elsewhere, a copy is required.

Source: Ruff FURB187 list-reverse-copy · `docs.astral.sh/ruff/rules/list-reverse-copy/`

YK108 info Gives advice **1.20× faster**

Membership test against a single item

x in [y] builds a list just to compare. Write x == y.

WHY DOES IT WASTE ENERGY?
A temporary list is created just to compare.

WHEN IS IT FINE?
For values that are not equal to themselves, such as NaN, 'in' also checks identity and == may differ.

WORK DONE **list + search → comparison**

WASTEFUL

```
def yonetici_mi(rol):
    return rol in ["yonetici"]
```

EFFICIENT

```
def yonetici_mi(rol):
    return rol == "yonetici"
```

MEASURED time 2.1 ms → 1.8 ms · memory 434.0 KB → 434.0 KB · same result ✓

Source: Ruff FURB171 single-item-membership-test · `docs.astral.sh/ruff/rules/single-item-membership-test/`

YK109 info Gives advice 7.41× faster

Converting to text to count bits

bin(x).count('1') turns the number into text first. On Python 3.10+, x.bit_count() counts directly.

WHY DOES IT WASTE ENERGY?

bin() turns the number into text and count scans it; bit_count counts with a single CPU instruction (popcount).

WHEN IS IT FINE?

bit_count does not exist before Python 3.10.

WORK DONE `string → one instruction`

WASTEFUL

```
def acik_bit_sayisi(izinler):
    return bin(izinler).count("1")
```

EFFICIENT

```
def acik_bit_sayisi(izinler):
    return izinler.bit_count()
```

MEASURED time 15.3 ms → 2.1 ms · memory 434.0 KB → 434.0 KB · same result ✓

Source: Ruff FURB161 bit-count · `docs.astral.sh/ruff/rules/bit-count/`

YK110 info Gives advice 1.41× faster

math.log(x, 2) or math.log(x, 10)

A log with a base computes two logarithms and a division. math.log2(x) and math.log10(x) do it in one step and more precisely.

WHY DOES IT WASTE ENERGY?

log(x, 2) computes two logarithms and a division; log2 is one function and more precise.

WHEN IS IT FINE?

If the base is a variable, log(x, base) is needed.

WORK DONE `2 logs + a division → 1`

WASTEFUL

```
import math


def bit_uzunlugu(n):
    return math.log(n, 2)
```

EFFICIENT

```
import math


def bit_uzunlugu(n):
    return math.log2(n)
```

MEASURED time 3.8 ms → 2.7 ms · memory 1.6 MB → 1.6 MB · same result ✓

The good version is more precise: values like math.log(1000, 10) can differ in the last digit; the comparison allows for that tiny difference.

Source: Ruff FURB163 redundant-log-base · `docs.astral.sh/ruff/rules/redundant-log-base/`

YK112 info Gives advice 1.26× faster

Consecutive append() calls

Instead of calling append() on the same list several times in a row, use one extend([...]).

WHY DOES IT WASTE ENERGY?

Every append is a separate method call; extend adds the elements in one call.

WHEN IS IT FINE?

Separate appends are natural if other work happens in between.

WORK DONE `n calls → 1`

WASTEFUL

```
def basliklar():
    satirlar = []
    satirlar.append("Ad")
    satirlar.append("Soyad")
    satirlar.append("Numara")
    return satirlar
```

EFFICIENT

```
def basliklar():
    return ["Ad", "Soyad", "Numara"]
```

MEASURED time 85 µs → 68 µs · memory 90.3 KB → 90.3 KB · same result ✓

Source: Ruff FURB113 repeated-append · `docs.astral.sh/ruff/rules/repeated-append/`

YK113 `info` `Engine fixes it` **`1.37× faster`**

Wrapping a generator in list(), set() or dict()

list(x for x in y) builds a generator and then drains it. Write a list, set or dict comprehension directly.

WHY DOES IT WASTE ENERGY?

A generator object is created and each element crosses a frame boundary; a comprehension builds the result directly.

WHEN IS IT FINE?

The difference is small, but the code is simpler too.

WORK DONE `generator + copy → comprehension`

WASTEFUL

```
def buyuk_harfli(adlar):
    return list(a.upper() for a in adlar)
```

EFFICIENT

```
def buyuk_harfli(adlar):
    return [a.upper() for a in adlar]
```

MEASURED time 1.2 ms → 864 µs · memory 1.1 MB → 1.1 MB · same result ✓

The engine's fix gave the same result too (1.55×).

Source: Ruff C400 / C401 / C402 · `docs.astral.sh/ruff/rules/unnecessary-generator-list/`

---

04 Text and regular expressions

7 rules

YK010 medium Gives advice

1.20× faster

String concatenation in a loop

Appending to a string with += in a loop can build a new string every step; with long text the work grows quadratically. Collect the pieces and use ''.join().

WHY DOES IT WASTE ENERGY?

Python strings are immutable; += may allocate a new string and copy the old one every time. CPython sometimes grows in place, but that is not guaranteed and fails if anything else references the string.

WHEN IS IT FINE?

When joining only a few pieces or the loop is short, the difference is negligible.

WORK DONE `O(n²) → O(n)`

WASTEFUL

```
def rapor_metni(satirlar):
    metin = ""
    for s in satirlar:
        metin += f"{s['ad']}: {s['tutar']} TL\n"
    return metin
```

EFFICIENT

```
def rapor_metni(satirlar):
    return "".join(f"{s['ad']}: {s['tutar']} TL\n" for s in
  satirlar)
```

MEASURED time 783 µs → 651 µs · memory 93.2 KB → 505.3 KB · same result ✓

CPython often grows a string in place on += when nothing else references it, so the gap can be small here. This is not guaranteed and does not exist in PyPy or other implementations.

Source: Pylint R1713 consider-using-join · `pylint.readthedocs.io/en/stable/user_guide/messages/refactor/consider-using-join.html`

YK069 medium Gives advice

no difference

Regular expression for plain text

The pattern has no special characters, so the regex engine is unnecessary. s.startswith(), in, str.replace() or str.split() do the same with less work.

WHY DOES IT WASTE ENERGY?

The regex engine compiles the pattern and matches with a state machine; str methods handle plain text with a direct C loop.

WHEN IS IT FINE?

If the pattern will grow into a real regex later, keeping re may help consistency.

WORK DONE `regex engine → direct comparison`

WASTEFUL

```
import re


def virgulle_bol(metin):
    return re.split(",", metin)
```

EFFICIENT

```
def virgulle_bol(metin):
    return metin.split(",")
```

MEASURED time 697 µs → 656 µs · memory 1.1 MB → 1.1 MB · same result ✓

Source: Python belgeleri: str yöntemleri · `docs.python.org/3/library/stdtypes.html#string-methods`

YK070 info Gives advice

no difference

Uncompiled regular expression in a loop

re.match/re.search looks the pattern up in the cache on every call. Compile it once with re.compile before the loop.

WHY DOES IT WASTE ENERGY?

The re module caches compiled patterns but still does a cache lookup and type checks on every call; a compiled object skips that.

WHEN IS IT FINE?

Because of the cache the difference is small; it matters in hot loops.

WORK DONE `n cache lookups → 0`

WASTEFUL

```
import re


def posta_kodlari(satirlar):
    return [re.search(r"\d{5}", s) for s in satirlar]
```

EFFICIENT

```
import re

POSTA_KODU = re.compile(r"\d{5}")


def posta_kodlari(satirlar):
    return [POSTA_KODU.search(s) for s in satirlar]
```

MEASURED time 14.2 ms → 13.2 ms · memory 2.5 MB → 2.5 MB · same result ✓

Source: Python belgeleri: re.compile · `docs.python.org/3/library/re.html#re.compile`

YK071 info Gives advice 1.60× faster

Repeated startswith() or endswith()

s.startswith('a') or s.startswith('b') makes a separate method call for each prefix. Pass them as a tuple: s.startswith(('a', 'b')).

WHY DOES IT WASTE ENERGY?
Each startswith call is a separate method lookup and call. Prefixes given as a tuple are tried in order inside C, in a single call.

WHEN IS IT FINE?
Cannot be merged when the conditions check different strings or do other work in between.

WORK DONE `k calls → 1`

WASTEFUL

```
def resim_mi(dosya_adi):
    return dosya_adi.endswith(".png") or
  dosya_adi.endswith(".jpg") or dosya_adi.endswith(".webp")
```

EFFICIENT

```
def resim_mi(dosya_adi):
    return dosya_adi.endswith((".png", ".jpg", ".webp"))
```

MEASURED time 3.5 ms → 2.2 ms · memory 434.0 KB → 434.0 KB · same result ✓

Source: Ruff PIE810 multiple-starts-ends-with · `docs.astral.sh/ruff/rules/multiple-starts-ends-with/`

YK072 info Gives advice no difference

find() just to test containment

find() computes a position; if you only need to know whether it is there, 'sub in text' is clearer and faster.

WHY DOES IT WASTE ENERGY?
find computes a position that is then compared; 'in' answers the containment question directly and reads better.

WHEN IS IT FINE?
If you need the position, find is the right tool.

WORK DONE `position + comparison → containment`

WASTEFUL

```
def hata_var_mi(log):
    return log.find("ERROR") != -1
```

EFFICIENT

```
def hata_var_mi(log):
    return "ERROR" in log
```

MEASURED time 738 µs → 727 µs · memory 28 B → 10 B · same result ✓

Same speed: both run the same C search. The gain is readability; the Python docs also recommend 'in' for containment.

Source: Python belgeleri: str.find · `docs.python.org/3/library/stdtypes.html#str.find`

YK073 info Gives advice 215× less memory

Reading a whole file then splitting it into lines

read().split('\n') or read().splitlines() keeps the whole file and the line list in memory together. Iterate the file line by line.

WHY DOES IT WASTE ENERGY?
read() loads the whole file as one string and split builds the list of lines as a second copy. Iterating the file object reads line by line through a buffer, so memory stays independent of file size. The Python tutorial recommends it as memory efficient, fast and simple.

WHEN IS IT FINE?
Reading the whole file is fine for small configuration files or when you need random access to lines.

WORK DONE `O(file) memory → O(line)`

WASTEFUL

```
def dolu_satir_sayisi(yol):
    with open(yol) as f:
        satirlar = f.read().split("\n")
    return sum(1 for s in satirlar if s.strip())
```

EFFICIENT

```
def dolu_satir_sayisi(yol):
    with open(yol) as f:
        return sum(1 for s in f if s.strip())
```

MEASURED time 9.9 ms → 12.1 ms · memory 9.6 MB → 45.5 KB · same result ✓

Similar time; the real gain is memory: the good version reads the file line by line.

Source: Python belgeleri: Dosya nesnelerinin yöntemleri (öğretici 7.2.1) · `docs.python.org/3/tutorial/inputoutput.html#methods-of-file-objects`

YK074 `info` `Gives advice` **1.69× faster**

Slicing to check a prefix

s[:3] == 'abc' slices out a new string first. s.startswith('abc') or s.endswith('abc') makes no copy.

WHY DOES IT WASTE ENERGY?

Slicing creates a new string object; startswith compares without a copy.

WHEN IS IT FINE?

For short prefixes the difference is small; readability improves.

WORK DONE `copy + comparison → comparison`

WASTEFUL

```
def https_mi(adres):
    return adres[:8] == "https://"
```

EFFICIENT

```
def https_mi(adres):
    return adres.startswith("https://")
```

MEASURED time 3.7 ms → 2.2 ms · memory 434.0 KB → 434.0 KB · same result ✓

Source: Python belgeleri: str.startswith · `docs.python.org/3/library/stdtypes.html#str.startswith`

---

05 Data science (pandas, NumPy)

17 rules

YK007 high Gives advice

914× faster

Iterating a pandas table row by row

iterrows, itertuples and apply(axis=1) process every row in a slow Python loop. Use vectorised pandas/numpy operations (column maths, merge, groupby).

WHY DOES IT WASTE ENERGY?

iterrows builds a separate Series object for every row and loops in the Python interpreter. Vectorised operations do the same calculation in compiled code in one pass.

WHEN IS IT FINE?

If each row calls an external service (an API), the loop cost is dominated by that call anyway.

WORK DONE `n steps in Python → vectorised in C`

WASTEFUL

```
def toplam_ciro(satislar):
    toplam = 0
    for _, satir in satislar.iterrows():
        toplam += satir["adet"] * satir["fiyat"]
    return toplam
```

EFFICIENT

```
def toplam_ciro(satislar):
    return (satislar["adet"] * satislar["fiyat"]).sum()
```

MEASURED time 34.4 ms → 38 µs · memory 50.9 KB → 50.1 KB · same result ✓

Source: pandas belgeleri: Enhancing performance · `pandas.pydata.org/docs/user_guide/enhancingperf.html`

YK018 high Gives advice

1.26× faster

Growing an array or table in a loop

pd.concat and np.append/concatenate/vstack copy all the data again on every call. Collect the parts and combine them once after the loop.

WHY DOES IT WASTE ENERGY?

NumPy arrays and DataFrames are contiguous blocks; every concat/append allocates a new block and copies all the data, which becomes quadratic in a loop.

WHEN IS IT FINE?

With only a few parts the difference is small.

WORK DONE `O(n²) → O(n)`

WASTEFUL

```
import pandas as pd


def dosyalari_birlestir(dosya_yollari):
    tablo = pd.DataFrame()
    for yol in dosya_yollari:
        parca = pd.read_csv(yol)
        tablo = pd.concat([tablo, parca])
    return tablo
```

EFFICIENT

```
import pandas as pd


def dosyalari_birlestir(dosya_yollari):
    return pd.concat([pd.read_csv(yol, usecols=["ad", "puan"])
  for yol in dosya_yollari])
```

MEASURED time 27.7 ms → 22.0 ms · memory 936.7 KB → 1.1 MB · same result ✓

Source: pandas belgeleri: concat · NumPy belgeleri: append · `pandas.pydata.org/docs/user_guide/merging.html`

YK027 high · Gives advice · **2,450× faster**

Element-by-element Python loop over a NumPy array

Looping over a NumPy array in Python is tens of times slower than vectorised operations. Work on the array directly: a * 2, np.where, np.sum.

WHY DOES IT WASTE ENERGY?
Every element access creates a Python object and runs in the interpreter; a vectorised operation does the same work in compiled code with SIMD instructions.

WHEN IS IT FINE?
If each step depends on the previous one (a recursive filter), vectorising can be hard; consider tools such as Numba.

WORK DONE `n steps in Python → vectorised in C`

WASTEFUL

```
import numpy as np


def normalize(dizi):
    sonuc = np.zeros(len(dizi))
    for i in range(len(dizi)):
        sonuc[i] = (dizi[i] - dizi.mean()) / dizi.std()
    return sonuc
```

EFFICIENT

```
def normalize(dizi):
    return (dizi - dizi.mean()) / dizi.std()
```

MEASURED time 39.9 ms → 16 µs · memory 48.2 KB → 48.1 KB · same result ✓

Source: NumPy belgeleri: Broadcasting ve vektörleştirme · `numpy.org/doc/stable/user/basics.broadcasting.html`

YK075 high · Gives advice · **not measured**

DataFrame.append in a loop

df = df.append(...) copies the whole table every step and was removed in pandas 2.0. Collect rows in a list and build one DataFrame after the loop.

WHY DOES IT WASTE ENERGY?
Every append copies the whole table into a new DataFrame; total copying for n rows grows quadratically. Removed in pandas 2.0.

WHEN IS IT FINE?
None; it no longer works in pandas 2.0+.

WORK DONE `O(n²) → O(n)`

WASTEFUL

```
import pandas as pd


def tabloya_ekle(satirlar):
    df = pd.DataFrame(columns=["ad", "puan"])
    for s in satirlar:
        df = df.append(s, ignore_index=True)
    return df
```

EFFICIENT

```
import pandas as pd


def tabloya_ekle(satirlar):
    return pd.DataFrame(satirlar, columns=["ad", "puan"])
```

NOT MEASURED DataFrame.append was removed in pandas 2.0, so the bad version does not run on current pandas. The working form of the same pattern is measured under YK076.

Source: pandas 2.0 sürüm notları: DataFrame.append kaldırıldı · `pandas.pydata.org/docs/whatsnew/v2.0.0.html`

YK076 high · Gives advice · **744× faster**

Adding rows with df.loc in a loop

df.loc[len(df)] = ... reallocates and copies the table on every insert. Collect the rows in a list and build the DataFrame at the end.

WHY DOES IT WASTE ENERGY?
Every row insert reallocates and copies the table to grow it.

WHEN IS IT FINE?
Small difference for a few rows.

WORK DONE `O(n²) → O(n)`

WASTEFUL

```
import pandas as pd


def tablo_olustur(satirlar):
    df = pd.DataFrame(columns=["ad", "puan"])
    for s in satirlar:
        df.loc[len(df)] = [s["ad"], s["puan"]]
    return df
```

EFFICIENT

```
import pandas as pd


def tablo_olustur(satirlar):
    return pd.DataFrame(satirlar, columns=["ad", "puan"])
```

MEASURED time 91.9 ms → 124 µs · memory 71.9 KB → 22.3 KB · same result ✓

Source: pandas belgeleri: Merge, join, concatenate · `pandas.pydata.org/docs/user_guide/merging.html`

YK077 medium Gives advice

90.4× faster

apply() for a vectorisable operation

apply(lambda x: x * 2) calls a Python function for every element. Operate on the column directly: df['a'] * 2.

WHY DOES IT WASTE ENERGY?

apply calls a Python lambda per element; a column operation does the same in C/NumPy in one pass.

WHEN IS IT FINE?

If the logic really needs per-row Python, apply may be required.

WORK DONE `n Python calls → vectorised`

WASTEFUL

```
def kdv_ekle(df):
    df["kdvli"] = df["fiyat"].apply(lambda x: x * 1.2)
    return df
```

EFFICIENT

```
def kdv_ekle(df):
    df["kdvli"] = df["fiyat"] * 1.2
    return df
```

MEASURED time 6.3 ms → 70 µs · memory 5.1 MB → 788.2 KB · same result ✓

Source: pandas belgeleri: Enhancing performance · `pandas.pydata.org/docs/user_guide/enhancingperf.html`

YK078 medium Gives advice

1.43× faster

apply() for a string operation

apply(lambda x: x.lower()) makes a Python call per cell. Use .str methods such as .str.lower().

WHY DOES IT WASTE ENERGY?

.str methods work in bulk and handle missing values (NaN) correctly; apply makes a Python call per cell.

WHEN IS IT FINE?

If the operation has no .str equivalent, apply may be needed.

WORK DONE `n Python calls → bulk`

WASTEFUL

```
def kucuk_harf(df):
    df["eposta"] = df["eposta"].apply(lambda x: x.lower())
    return df
```

EFFICIENT

```
def kucuk_harf(df):
    df["eposta"] = df["eposta"].str.lower()
    return df
```

MEASURED time 5.4 ms → 3.8 ms · memory 5.4 MB → 5.4 MB · same result ✓

Source: pandas belgeleri: Working with text data · `pandas.pydata.org/docs/user_guide/text.html`

YK079 info Gives advice

205× faster

Expecting speed from np.vectorize

np.vectorize is a convenience; per the NumPy docs it is essentially a for loop. Use real vectorised operations or np.where.

WHY DOES IT WASTE ENERGY?

np.vectorize calls the Python function once per element; per the NumPy docs it is essentially a for loop.

WHEN IS IT FINE?

Acceptable for readability, but do not expect speed.

WORK DONE `n Python calls → vectorised`

WASTEFUL

```
import numpy as np


def notlari_harflendir(notlar):
    harf = np.vectorize(lambda n: "AA" if n >= 90 else "BB")
    return harf(notlar)
```

EFFICIENT

```
import numpy as np


def notlari_harflendir(notlar):
    return np.where(notlar >= 90, "AA", "BB")
```

MEASURED time 7.8 ms → 38 µs · memory 1.1 MB → 441.2 KB · same result ✓

Source: NumPy belgeleri: numpy.vectorize · `numpy.org/doc/stable/reference/generated/numpy.vectorize.html`

YK080 medium Gives advice **13.8× faster**

Built-in sum(), min() or max() on a NumPy array

Python's sum/min/max walk the array element by element as Python objects. arr.sum(), arr.min() and arr.max() run at C speed.

WHY DOES IT WASTE ENERGY?

Python's sum turns the array into NumPy scalars one by one and adds them in the interpreter; arr.sum() adds in C with SIMD.

WHEN IS IT FINE?

No difference for tiny arrays.

WORK DONE **Python loop → C loop**

WASTEFUL

```
import numpy as np


def ortalama(n):
    olcumler = np.random.rand(n)
    return sum(olcumler) / n
```

EFFICIENT

```
import numpy as np


def ortalama(n):
    olcumler = np.random.rand(n)
    return olcumler.sum() / n
```

MEASURED time 11.6 ms → 847 µs · memory 1.5 MB → 1.5 MB · same result ✓

Source: NumPy belgeleri: ndarray.sum · numpy.org/doc/stable/reference/generated/numpy.ndarray.sum.html

---

YK081 medium Gives advice **2.00× faster**

Creating a constant array or tensor in a loop

np.array/torch.tensor is created with the same content on every iteration. Create it once before the loop.

WHY DOES IT WASTE ENERGY?

Memory is allocated and data copied every step; a constant array can be created once.

WHEN IS IT FINE?

If the array is modified in the loop, a fresh one may be needed each step.

WORK DONE **n allocations → 1**

WASTEFUL

```
import numpy as np


def donustur(noktalar):
    sonuc = []
    for p in noktalar:
        matris = np.array([[0, -1], [1, 0]])
        sonuc.append(matris @ p)
    return sonuc
```

EFFICIENT

```
import numpy as np

DONDUR = np.array([[0, -1], [1, 0]])


def donustur(noktalar):
    return [DONDUR @ p for p in noktalar]
```

MEASURED time 7.6 ms → 3.8 ms · memory 666.4 KB → 666.3 KB · same result ✓

Source: NumPy belgeleri: Array creation · numpy.org/doc/stable/user/basics.creation.html

---

YK082 info Gives advice **1.20× slower**

Assignment through chained indexing

df['col'][i] = x may create an intermediate copy and lose the change. Use df.loc[i, 'col'] = x.

WHY DOES IT WASTE ENERGY?

The first indexing may return a copy; the assignment goes to the copy, the table does not change (SettingWithCopyWarning), and the copy costs memory.

WHEN IS IT FINE?

Behaviour changes with pandas 3.0 copy-on-write; .loc is correct in every version.

WORK DONE **intermediate copy → in-place assignment**

WASTEFUL

```
def indirim_uygula(df, satir):
    df["fiyat"][satir] = df["fiyat"][satir] * 0.9
    return df
```

EFFICIENT

```
def indirim_uygula(df, satir):
    df.loc[satir, "fiyat"] = df.loc[satir, "fiyat"] * 0.9
    return df
```

MEASURED time 58 µs → 70 µs · memory 3.4 KB → 10.7 KB · same result ✓

This rule is really about correctness: if the chained assignment hits a copy, the change is lost. In pandas 3.0 (copy-on-write) the bad version never changes the table.

Source: pandas belgeleri: Returning a view versus a copy · pandas.pydata.org/docs/user_guide/indexing.html#returning-a-view-versus-a-copy

---

YK083 info Gives advice **1.96× faster**

Aggregating with groupby().apply()

groupby().apply() calls a Python function for every group. Use agg() for aggregates and transform() for per-row results.

WHY DOES IT WASTE ENERGY?

apply calls a Python function per group and stitches results; agg and transform use optimised paths.

WHEN IS IT FINE?

If each group really needs complex logic, apply may be needed.

WORK DONE **k Python calls → optimised aggregation**

WASTEFUL

```
def sehir_ortalamasi(df):
    return df.groupby("sehir").apply(lambda g:
  g["tutar"].mean())
```

EFFICIENT

```
def sehir_ortalamasi(df):
    return df.groupby("sehir")["tutar"].mean()
```

MEASURED time 3.5 ms → 1.8 ms · memory 2.3 MB → 1.8 MB · same result ✓

Source: pandas belgeleri: Group by (agg, transform) · pandas.pydata.org/docs/user_guide/groupby.html

---

YK084 info Gives advice **1.31× faster**

Reading every column with read_csv

Without usecols every column is read, usually with memory-hungry types. Pass usecols and dtype.

WHY DOES IT WASTE ENERGY?

Every column is parsed and text columns are stored as Python objects; memory can be several times what is needed.

WHEN IS IT FINE?

No difference for small files or when every column is needed.

WORK DONE **every column → the needed ones**

WASTEFUL

```
import pandas as pd


def satislari_oku(yol):
    df = pd.read_csv(yol)
    return df["tutar"].sum()
```

EFFICIENT

```
import pandas as pd


def satislari_oku(yol):
    df = pd.read_csv(yol, usecols=["tutar"], dtype={"tutar":
  "int64"})
    return df["tutar"].sum()
```

MEASURED time 24.3 ms → 18.6 ms · memory 7.0 MB → 2.0 MB · same result ✓

Source: pandas belgeleri: Scaling to large datasets · pandas.pydata.org/docs/user_guide/scale.html

---

YK085 info Gives advice **5.03× faster**

CSV for large data

CSV is uncompressed text and every value is parsed again on read. For large tables use a compressed columnar format such as Parquet.

WHY DOES IT WASTE ENERGY?

CSV must be parsed as text on every read, loses types and is uncompressed. Parquet is columnar, typed and compressed, and far cheaper to read.

WHEN IS IT FINE?

CSV may be needed to share with other systems; for small files the difference is negligible.

WORK DONE **text parsing → columnar read**

WASTEFUL

```
def disari_aktar(df):
    df.to_csv("buyuk_tablo.csv", index=False)
```

EFFICIENT

```
def disari_aktar(df):
    df.to_parquet("buyuk_tablo.parquet", index=False)
```

MEASURED time 52.5 ms → 10.4 ms · memory 7.5 MB → 16.4 KB · file 1.0 MB → 605.6 KB · same result ✓

Outputs were compared by reading the files back. Parquet also takes less disk space and later reads are much faster.

Source: Green Software Foundation: Compress stored data · pandas: Scaling to large datasets · pandas.pydata.org/docs/user_guide/scale.html#use-efficient-datatypes

---

YK086 info Gives advice **1.69× faster**

Converting a NumPy array with list()

list(arr) turns each element into a separate NumPy scalar. arr.tolist() is faster; usually staying with the array is best.

WHY DOES IT WASTE ENERGY?

list() wraps each element in a separate NumPy scalar; tolist() converts straight to Python numbers.

WHEN IS IT FINE?

Negligible for small arrays.

WORK DONE **NumPy scalars → Python numbers**

WASTEFUL

```
import numpy as np


def listeye(n):
    dizi = np.arange(n)
    return list(dizi)
```

EFFICIENT

```
import numpy as np


def listeye(n):
    return np.arange(n).tolist()
```

MEASURED time 2.7 ms → 1.6 ms · memory 3.8 MB → 4.6 MB · same result ✓

Source: NumPy belgeleri: ndarray.tolist · numpy.org/doc/stable/reference/generated/numpy.ndarray.tolist.html

---

YK087 medium Gives advice **20.5× faster**

Filtering a DataFrame in a loop

df[df['k'] == value] in a loop scans the whole table every step. groupby('k') gives every group in one pass.

WHY DOES IT WASTE ENERGY?

Each filter scans the whole table and builds a boolean mask; k groups mean k full passes. groupby groups in one pass.

WHEN IS IT FINE?

Small difference when filtering only one or two values.

WORK DONE **k passes → 1**

WASTEFUL

```
def sehir_toplamlari(df, sehirler):
    sonuc = {}
    for s in sehirler:
        sonuc[s] = df[df["sehir"] == s]["tutar"].sum()
    return sonuc
```

EFFICIENT

```
def sehir_toplamlari(df, sehirler):
    toplamlar = df.groupby("sehir")["tutar"].sum()
    return {s: toplamlar.get(s, 0) for s in sehirler}
```

MEASURED time 34.5 ms → 1.7 ms · memory 146.9 KB → 1.8 MB · same result ✓

Source: pandas belgeleri: Group by · pandas.pydata.org/docs/user_guide/groupby.html

---

YK111 info Gives advice **28.2× less memory**

nunique() to check for a constant series

nunique() counts every distinct value. To check that a series is constant, (s == s.iloc[0]).all() can stop at the first difference.

WHY DOES IT WASTE ENERGY?

nunique builds a hash table of every value; a constant check can stop at the first different value.

WHEN IS IT FINE?

If the number of distinct values is needed elsewhere too, nunique is right.

WORK DONE **full hash table → up to the first difference**

WASTEFUL

```
def tek_sehir_mi(df):
    return df["sehir"].nunique() == 1
```

EFFICIENT

```
def tek_sehir_mi(df):
    sehir = df["sehir"]
    return bool((sehir == sehir.iloc[0]).all())
```

MEASURED time 2.9 ms → 2.8 ms · memory 2.8 MB → 101.0 KB · same result ✓

Every value is the same, so both versions scan the whole series and take the same time. The gain is memory: nunique builds a hash table of every value.

Source: Ruff PD101 pandas-nunique-constant-series-check · docs.astral.sh/ruff/rules/pandas-nunique-constant-series-check/

---

06 Machine learning

4 rules

YK088 high Gives advice

77.2× faster

Predicting one sample at a time in a loop

Calling the model separately for each sample loses the efficiency of batching. Collect the samples and send them to predict() together.

WHY DOES IT WASTE ENERGY?
Every predict call pays for input validation, allocation and (on GPU) transfer; a batched call pays that fixed cost once and computes in a vectorised way.

WHEN IS IT FINE?
Batching may be impossible for real-time single requests (one prediction per web request).

WORK DONE **n fixed costs → 1**

WASTEFUL

```
def siniflandir(model, ornekler):
    return [model.predict([x])[0] for x in ornekler]
```

EFFICIENT

```
def siniflandir(model, ornekler):
    return list(model.predict(ornekler))
```

MEASURED time 10.1 ms → 132 µs · memory 11.4 KB → 17.5 KB · same result ✓

Source: scikit-learn ve Keras belgeleri: toplu tahmin · scikit-learn.org/stable/computing/computational_performance.html

YK089 info Gives advice

not measured

Pulling values off the GPU inside a loop

.item(), .cpu() and .numpy() force the CPU to wait for the GPU on every call (synchronisation). Read values every few steps instead.

WHY DOES IT WASTE ENERGY?
The GPU works asynchronously from a queue; .item() forces a wait for the result and drains the queue.

WHEN IS IT FINE?
Unavoidable if the value is really needed every step (early stopping); read it rarely.

WORK DONE **sync every step → rarely**

WASTEFUL

```
def egit(model, veri, kayip_fn, optimizer):
    kayiplar = []
    for x, y in veri:
        kayip = kayip_fn(model(x), y)
        kayip.backward()
        optimizer.step()
        kayiplar.append(kayip.item())
    return kayiplar
```

EFFICIENT

```
import torch


def egit(model, veri, kayip_fn, optimizer):
    kayiplar = []
    for x, y in veri:
        kayip = kayip_fn(model(x), y)
        kayip.backward()
        optimizer.step()
        kayiplar.append(kayip.detach())
    return torch.stack(kayiplar).tolist()
```

NOT MEASURED Needs PyTorch and a GPU. There is no synchronisation wait on the CPU, so the gain only shows on a GPU.

Source: PyTorch Performance Tuning Guide · docs.pytorch.org/tutorials/recipes/recipes/tuning_guide.html

YK090 medium Gives advice

not measured

Moving the same object to the device in a loop

An object that does not depend on the loop is moved with .to(device) or .cuda() on every step. Move it once before the loop.

WHY DOES IT WASTE ENERGY?
Every move means copying data between CPU and GPU and synchronising.

WHEN IS IT FINE?
If the object differs every step (a new batch), it must be moved.

WORK DONE **n transfers → 1**

WASTEFUL

```
def tahmin(model, veri, device):
    sonuclar = []
    for x in veri:
        model.to(device)
        sonuclar.append(model(x.to(device)))
    return sonuclar
```

EFFICIENT

```
def tahmin(model, veri, device):
    model.to(device)
    return [model(x.to(device)) for x in veri]
```

NOT MEASURED Needs PyTorch and a GPU. The gain is removing the CPU→GPU copy on every step.

Source: PyTorch Performance Tuning Guide · docs.pytorch.org/tutorials/recipes/recipes/tuning_guide.html

YK091 medium Gives advice not measured

Computing gradients during inference

The model is in eval() mode but calls are not inside torch.no_grad() or torch.inference_mode(), so memory and compute are spent on gradients.

WHY DOES IT WASTE ENERGY?

Gradient tracking keeps every operation's intermediate results for backpropagation; inference does not need them.

WHEN IS IT FINE?

If gradients are needed during inference (explainability, adversarial tests), do not use no_grad.

WORK DONE `gradient memory → none`

WASTEFUL

```
def degerlendir(model, veri):
    model.eval()
    dogru = 0
    for x, y in veri:
        cikti = model(x)
        dogru += int((cikti.argmax(1) == y).sum())
    return dogru
```

EFFICIENT

```
import torch


def degerlendir(model, veri):
    model.eval()
    dogru = 0
    with torch.inference_mode():
        for x, y in veri:
            cikti = model(x)
            dogru += int((cikti.argmax(1) == y).sum())
    return dogru
```

NOT MEASURED Needs PyTorch. The gain depends on the model; the memory kept for gradients during inference disappears entirely.

Source: PyTorch Performance Tuning Guide: torch.no_grad · docs.pytorch.org/tutorials/recipes/recipes/tuning_guide.html

07 Network and cloud

7 rules

YK008 medium Gives advice

not measured

HTTP request without a session in a loop

Every requests call opens a new connection and TLS handshake. Open requests.Session() before the loop and reuse the connection.

WHY DOES IT WASTE ENERGY?

Each request without a session opens a new TCP connection and TLS handshake: several round trips and cryptography per request. A Session keeps the connection open and reuses it.

WHEN IS IT FINE?

If every request goes to a different host, the connection cannot be reused.

WORK DONE `n handshakes → 1`

WASTEFUL

```
import requests


def fiyatlari_cek(urun_kodlari):
    return
  [requests.get(f"https://api.ornek.com.tr/fiyat/{kod}").json()
  for kod in urun_kodlari]
```

EFFICIENT

```
import requests


def fiyatlari_cek(urun_kodlari):
    with requests.Session() as oturum:
        return
  [oturum.get(f"https://api.ornek.com.tr/fiyat/{kod}",
  timeout=10).json() for kod in urun_kodlari]
```

NOT MEASURED Needs HTTPS requests to a real server. A session removes the per-request TCP and TLS handshake (usually 1–2 round trips).

Source: requests belgeleri: Session Objects (bağlantı havuzu) · `requests.readthedocs.io/en/latest/user/advanced/#session-objects`

YK014 high Gives advice

186× faster

Creating a connection, client or pool in a loop

Opening a database connection, HTTP session, cloud client or thread pool is expensive (handshakes, authentication). Create it once before the loop.

WHY DOES IT WASTE ENERGY?

Connecting goes through DNS, TCP, TLS and authentication; it is often more expensive than the query itself. One connection or a pool pays that cost once.

WHEN IS IT FINE?

If each step connects to a different target, reuse is impossible.

WORK DONE `n connections → 1`

WASTEFUL

```
import sqlite3


def kayitlari_yaz(kayitlar):
    for k in kayitlar:
        baglanti = sqlite3.connect("veri.db")
        baglanti.execute("INSERT INTO log VALUES (?)", (k,))
        baglanti.commit()
        baglanti.close()
```

EFFICIENT

```
import sqlite3


def kayitlari_yaz(kayitlar):
    baglanti = sqlite3.connect("veri.db")
    with baglanti:
        baglanti.executemany("INSERT INTO log VALUES (?)",
  [(k,) for k in kayitlar])
    baglanti.close()
```

MEASURED time 113.5 ms → 609 µs · memory 3.8 KB → 4.9 KB · same result ✓

Measured with an SQLite file on disk. Since every connection has to close its own transaction, the gap includes the drop in commits as well.

Source: SQLAlchemy: Connection Pooling · requests: Session · `docs.sqlalchemy.org/en/20/core/pooling.html`

YK092 medium Gives advice not measured

Object-storage request per item in a loop

Every get_object/head_object is a separate network request. Run them concurrently, use listing results or read in bulk.

WHY DOES IT WASTE ENERGY?

Every object request is a separate HTTP request; latency and request fees multiply by the count.

WHEN IS IT FINE?

If objects are needed separately, running requests concurrently still gives a big gain.

WORK DONE `n sequential requests → concurrent/batched`

WASTEFUL

```
def raporlari_indir(s3, anahtarlar):
    return [s3.get_object(Bucket="raporlar", Key=k)
  ["Body"].read() for k in anahtarlar]
```

EFFICIENT

```
from concurrent.futures import ThreadPoolExecutor


def raporlari_indir(s3, anahtarlar):
    def indir(anahtar):
        return s3.get_object(Bucket="raporlar", Key=anahtar)
  ["Body"].read()

    with ThreadPoolExecutor(max_workers=16) as havuz:
        return list(havuz.map(indir, anahtarlar))
```

NOT MEASURED Needs a cloud storage (S3) account. The gain comes from overlapping sequential request latencies.

Source: Green Software Foundation: Keep request counts low · AWS S3 performans kılavuzu · `docs.aws.amazon.com/AmazonS3/latest/userguide/optimizing-performance.html`

YK093 medium Gives advice not measured

One Redis command per iteration

Every command is a separate round trip to the server. Send them together with pipeline() or mget/mset.

WHY DOES IT WASTE ENERGY?

Every command is a network round trip; a pipeline sends commands together and reads replies in bulk.

WHEN IS IT FINE?

If one result decides the next command, a pipeline cannot be used.

WORK DONE `n round trips → 1`

WASTEFUL

```
import redis


def sepetleri_getir(kullanicilar):
    r = redis.Redis()
    return [r.get(f"sepet:{k}") for k in kullanicilar]
```

EFFICIENT

```
import redis


def sepetleri_getir(kullanicilar):
    r = redis.Redis()
    return r.mget([f"sepet:{k}" for k in kullanicilar])
```

NOT MEASURED Needs a Redis server. The gain is in round trips: n → 1.

Source: Green Software Foundation: Keep request counts low · Redis: Pipelining · `redis.io/docs/latest/develop/use/pipelining/`

YK094 medium Gives advice not measured

urllib requests in a loop

urllib.request.urlopen opens a new connection for every request. Use requests.Session or httpx.Client, which reuse connections.

WHY DOES IT WASTE ENERGY?

urllib opens and closes a new connection for each request; there is no connection pool.

WHEN IS IT FINE?

Negligible in one-off scripts.

WORK DONE `n connections → 1`

WASTEFUL

```
import urllib.request


def sayfalari_indir(adresler):
    return [urllib.request.urlopen(a).read() for a in adresler]
```

EFFICIENT

```
import requests


def sayfalari_indir(adresler):
    with requests.Session() as oturum:
        return [oturum.get(a, timeout=10).content for a in
  adresler]
```

NOT MEASURED Needs requests to a real server. The gain is removing connection setup per request.

Source: requests belgeleri: Session Objects · `requests.readthedocs.io/en/latest/user/advanced/#session-objects`

YK095 medium Gives advice not measured

Synchronous HTTP call inside an async function

requests blocks the event loop; nothing else progresses meanwhile. Use httpx.AsyncClient or aiohttp.

WHY DOES IT WASTE ENERGY?

The event loop runs on a single thread; until the blocking request finishes, every other task waits.

WHEN IS IT FINE?

Wrapping the call in asyncio.to_thread avoids the blocking.

WORK DONE **serial waiting → concurrent**

WASTEFUL

```
import requests


async def hava_durumu(sehir):
    return requests.get(f"https://api.ornek.com/hava/{sehir}",
  timeout=5).json()
```

EFFICIENT

```
import httpx


async def hava_durumu(sehir):
    async with httpx.AsyncClient(timeout=5) as istemci:
        yanit = await
  istemci.get(f"https://api.ornek.com/hava/{sehir}")
        return yanit.json()
```

NOT MEASURED Needs requests to a real server. The gain is not in one call's duration but in other waiting tasks being able to progress.

Source: Python belgeleri: asyncio, engelleyen kod · docs.python.org/3/library/asyncio-dev.html#running-blocking-code

---

YK097 info Gives advice not measured

HTTP request without a timeout

Without a timeout an unresponsive server can block the request forever, keeping the process and connection open for nothing. Add timeout=....

WHY DOES IT WASTE ENERGY?

A request to an unresponsive server can wait forever, keeping the process, connection and memory open for nothing.

WHEN IS IT FINE?

None; give every network request a timeout.

WORK DONE **unbounded → bounded**

WASTEFUL

```
import requests


def kur_getir():
    return requests.get("https://api.ornek.com/kur").json()
```

EFFICIENT

```
import requests


def kur_getir():
    return requests.get("https://api.ornek.com/kur",
  timeout=10).json()
```

NOT MEASURED Needs an unresponsive server. A timeout bounds an otherwise infinite wait.

Source: requests belgeleri: Timeouts · requests.readthedocs.io/en/latest/user/advanced/#timeouts

---

08 Files and I/O

8 rules

YK012 high Gives advice

not measured

Reloading the same file or model in a loop

The same file or model is read from disk on every iteration. Load it once before the loop and reuse it.

WHY DOES IT WASTE ENERGY?
Reading and parsing the file or model from disk on every step produces the same result again; with large models each load is seconds and hundreds of MB.

WHEN IS IT FINE?
If the file really changes during the loop (live monitoring), it has to be re-read.

WORK DONE `n loads → 1`

WASTEFUL

```
import joblib


def tahmin_et(ornekler):
    sonuclar = []
    for x in ornekler:
        model = joblib.load("model.pkl")
        sonuclar.append(model.predict([x])[0])
    return sonuclar
```

EFFICIENT

```
import joblib


def tahmin_et(ornekler):
    model = joblib.load("model.pkl")
    return list(model.predict(ornekler))
```

NOT MEASURED Needs a trained model file. The gain is model size × loop length of disk reads and parsing.

Source: Green Software Foundation: Cache static data · `patterns.greensoftware.foundation/development`

YK019 info Gives advice

6.02× faster

Pre-formatted log message in a loop

A log message built with an f-string or format() is formatted on every iteration even when that level is off. Use logger.debug('%s', x).

WHY DOES IT WASTE ENERGY?
An f-string is evaluated before the call even when the log level is off. With %s formatting, the message is built only if it is actually emitted.

WHEN IS IT FINE?
If the message is at a level that is always emitted (error), there is no difference.

WORK DONE `formatting every step → only when needed`

WASTEFUL

```
import logging

logger = logging.getLogger(__name__)


def isle(kayitlar):
    for k in kayitlar:
        logger.debug(f"İşleniyor: {k}")
        k.kaydet()
```

EFFICIENT

```
import logging

logger = logging.getLogger(__name__)


def isle(kayitlar):
    for k in kayitlar:
        logger.debug("İşleniyor: %s", k)
        k.kaydet()
```

MEASURED time 3.6 ms → 598 µs · memory 442 B → 48 B · same result ✓

The log level is WARNING; debug messages are not written, yet the bad version still formats every one of them.

Source: Pylint W1203 logging-fstring-interpolation · `docs.python.org/3/howto/logging.html#optimization`

YK020 info Gives advice **132× less memory**

Reading a whole file with readlines()

readlines() reads the entire file into memory first. Iterate over the file object directly: for line in file.

WHY DOES IT WASTE ENERGY?

readlines loads the whole file at once and builds a list of lines; memory grows with the file. The file object reads line by line with a buffer.

WHEN IS IT FINE?

If you need random access to lines or several passes, a list may be needed.

WORK DONE **O(file) memory → O(line)**

WASTEFUL

```
def hata_say(log_yolu):
    sayi = 0
    with open(log_yolu) as f:
        for satir in f.readlines():
            if "ERROR" in satir:
                sayi += 1
    return sayi
```

EFFICIENT

```
def hata_say(log_yolu):
    with open(log_yolu) as f:
        return sum(1 for satir in f if "ERROR" in satir)
```

MEASURED time 6.1 ms → 5.6 ms · memory 5.9 MB → 45.6 KB · same result ✓

Similar time; the real gain is memory: the good version reads the file line by line.

Source: Ruff FURB129 readlines-in-for · docs.astral.sh/ruff/rules/readlines-in-for/

YK098 medium Gives advice **27.6× faster**

flush() inside a loop

flush() empties the buffer and makes a system call every step. Let buffering work and flush once after the loop if needed.

WHY DOES IT WASTE ENERGY?

Buffered writing merges many small writes into one system call; flush breaks that on every step.

WHEN IS IT FINE?

If output must appear immediately (live logs), flushing every so often is enough.

WORK DONE **n system calls → few**

WASTEFUL

```
def kaydet(dosya, satirlar):
    for s in satirlar:
        dosya.write(s + "\n")
        dosya.flush()
```

EFFICIENT

```
def kaydet(dosya, satirlar):
    dosya.writelines(s + "\n" for s in satirlar)
```

MEASURED time 12.6 ms → 457 µs · memory 360 B → 46.8 KB · same result ✓

Source: Python belgeleri: io (tamponlu akışlar) · docs.python.org/3/library/io.html

---

YK099 high Gives advice **41.4× faster**

os.fsync() inside a loop

fsync waits for the disk to finish writing physically; in a loop it is very expensive. Write in bulk and sync once.

WHY DOES IT WASTE ENERGY?

fsync waits for the disk to confirm data is durable; even on an SSD it takes milliseconds.

WHEN IS IT FINE?

Required when each record must be durable on its own (a journal); writing in bulk and syncing once is usually enough.

WORK DONE **n disk syncs → 1**

WASTEFUL

```
import os


def guvenli_yaz(f, kayitlar):
    for k in kayitlar:
        f.write(k)
        f.flush()
        os.fsync(f.fileno())
```

EFFICIENT

```
import os


def guvenli_yaz(f, kayitlar):
    f.writelines(kayitlar)
    f.flush()
    os.fsync(f.fileno())
```

MEASURED time 2.8 ms → 68 µs · memory 360 B → 6.0 KB · same result ✓

Source: SQLite SSS ve POSIX: fsync maliyeti · www.sqlite.org/faq.html#q19

YK100 info Gives advice

3.56× faster

print() inside a loop

print() writes to the output stream on every call; in large loops this is real I/O load. Collect results and print once, or use logging levels.

WHY DOES IT WASTE ENERGY?

Every print formats and writes; writing to a terminal is especially slow and is often flushed line by line.

WHEN IS IT FINE?

Fine in short scripts and during development.

WORK DONE `n writes → 1`

WASTEFUL

```
def isle(kayitlar):
    for k in kayitlar:
        print("İşleniyor:", k)
```

EFFICIENT

```
def isle(kayitlar):
    print("\n".join(f"İşleniyor: {k}" for k in kayitlar))
```

MEASURED time 1.4 ms → 399 µs · memory 252.2 KB → 740.1 KB · same result ✓

Output was redirected to an in-memory buffer. Writing to a terminal or a file makes every call more expensive.

Source: Python belgeleri: logging HOWTO · `docs.python.org/3/howto/logging.html`

YK101 medium Gives advice

280× faster

Rescanning a directory in a loop

os.listdir/glob rescans the same directory on disk every step. Get the listing once before the loop.

WHY DOES IT WASTE ENERGY?

Listing a directory reads file-system metadata; scanning the same directory every step produces the same result again.

WHEN IS IT FINE?

If the directory changes during the loop (waiting for new files), re-scanning is needed.

WORK DONE `n scans → 1`

WASTEFUL

```
import os


def yeni_dosyalar(gorulen):
    yeniler = []
    for g in gorulen:
        if g not in os.listdir("gelen"):
            yeniler.append(g)
    return yeniler
```

EFFICIENT

```
import os


def yeni_dosyalar(gorulen):
    mevcut = set(os.listdir("gelen"))
    return [g for g in gorulen if g not in mevcut]
```

MEASURED time 56.8 ms → 203 µs · memory 19.3 KB → 28.1 KB · same result ✓

Source: Python belgeleri: os.scandir · `docs.python.org/3/library/os.html#os.scandir`

YK105 info Gives advice

no difference

write() one item at a time in a loop

Every write() is a separate call. f.writelines(lines) does the same in one call.

WHY DOES IT WASTE ENERGY?

Every write is a method call; writelines writes an iterable in one call.

WHEN IS IT FINE?

A loop may be needed if other work happens between lines.

WORK DONE `n calls → 1`

WASTEFUL

```
def kaydet(yol, satirlar):
    with open(yol, "w") as f:
        for s in satirlar:
            f.write(s)
```

EFFICIENT

```
def kaydet(yol, satirlar):
    with open(yol, "w") as f:
        f.writelines(satirlar)
```

MEASURED time 2.0 ms → 2.2 ms · memory 51.2 KB → 51.2 KB · same result ✓

No difference on a buffered file: write calls already go to an in-memory buffer. The gain is simplicity.

Source: Ruff FURB122 for-loop-writes · `docs.astral.sh/ruff/rules/for-loop-writes/`

---

09 Concurrency and waiting

7 rules

YK005 medium Gives advice

not measured

Busy polling

Spinning in a loop to re-check wastes CPU and network. Use events (webhooks, queues, notifications) or wait longer.

WHY DOES IT WASTE ENERGY?
A loop that keeps checking wakes the CPU and sends network requests even when there is nothing to do, and loads the other side for nothing.

WHEN IS IT FINE?
With legacy systems that have no event mechanism, polling at long intervals (minutes) is acceptable.

WORK DONE **constant wake-ups → only when an event arrives**

WASTEFUL

```
import time


def siparis_bekle(api):
    while True:
        if api.yeni_siparis_var_mi():
            return api.siparisi_al()
        time.sleep(0.1)
```

EFFICIENT

```
import queue


def siparis_bekle(siparisler: queue.Queue):
    # Webhook ya da kuyruk: sipariş gelince uyanır, arada
  işlemci ve ağ boştadır.
    return siparisler.get()
```

NOT MEASURED Waits on an external service (API). The gain is in request count: a loop polling every 0.1 s sends 600 requests a minute, an event-driven design sends one.

Source: Green Software Foundation: Use Asynchronous network calls instead of synchronous · patterns.greensoftware.foundation/development

---

YK015 high Gives advice

544× less CPU

Waiting with an empty loop (busy wait)

A loop with an empty body keeps one CPU core at 100% until the condition changes. Wait with threading.Event, a queue or time.sleep.

WHY DOES IT WASTE ENERGY?
An empty loop never lets the CPU sleep; the core spins at full power until the condition changes, spent directly as battery, heat and electricity.

WHEN IS IT FINE?
It can be deliberate in microsecond-level low-level concurrency code; almost never in application code.

WORK DONE **100% of a core → ~0%**

WASTEFUL

```
def hazir_olmasini_bekle(is_parcacigi):
    while not is_parcacigi.bitti:
        pass
    return is_parcacigi.sonuc
```

EFFICIENT

```
def hazir_olmasini_bekle(is_parcacigi):
    is_parcacigi.join()
    return is_parcacigi.sonuc
```

MEASURED time 31.3 ms → 25.0 ms · CPU 31.3 ms → 58 µs · memory 144 B → 144 B · same result ✓

Both versions wait 20 ms for the work to finish, so the duration is the same. The difference is the idle work the CPU does meanwhile, so the ratio comes from CPU time.

Source: Python belgeleri: threading.Event · docs.python.org/3/library/threading.html#event-objects

---

YK030 medium Gives advice

not measured

Starting a process in a loop

Every subprocess or os.system call starts a new operating-system process. Batch the work or use a Python library that does the same job.

WHY DOES IT WASTE ENERGY?

Every process start means fork/exec, memory mapping and loading a program; for short jobs that often costs more than the work itself.

WHEN IS IT FINE?

Unavoidable when the external program has no Python equivalent and cannot take a batch.

WORK DONE `n processes → 1`

WASTEFUL

```
import subprocess


def resimleri_kucult(dosyalar):
    for d in dosyalar:
        subprocess.run(["convert", d, "-resize", "50%",
  "kucuk_" + d])
```

EFFICIENT

```
import subprocess


def resimleri_kucult(dosyalar):
    # Tek süreç bütün dosyaları işler (ya da Pillow ile hiç
  süreç açmadan).
    subprocess.run(["mogrify", "-resize", "50%", "-path",
  "kucuk", *dosyalar], check=True)
```

NOT MEASURED Needs an external program (ImageMagick). Each process start takes milliseconds; a single process pays that once.

Source: Python belgeleri: subprocess · `docs.python.org/3/library/subprocess.html`

YK096 medium Gives advice

not measured

time.sleep inside an async function

time.sleep stops the whole event loop. Use await asyncio.sleep(...).

WHY DOES IT WASTE ENERGY?

time.sleep puts the thread to sleep, and the event loop stops with it.

WHEN IS IT FINE?

None; always use asyncio.sleep in async code.

WORK DONE `loop stalls → loop keeps running`

WASTEFUL

```
import time


async def tekrar_dene(istek):
    for _ in range(3):
        if await istek():
            return True
        time.sleep(1)
    return False
```

EFFICIENT

```
import asyncio


async def tekrar_dene(istek):
    for _ in range(3):
        if await istek():
            return True
        await asyncio.sleep(1)
    return False
```

NOT MEASURED The gain is not in one call's duration but in other tasks on the event loop progressing during the wait.

Source: Python belgeleri: asyncio.sleep · `docs.python.org/3/library/asyncio-task.html#asyncio.sleep`

YK102 medium Gives advice

3.26× less CPU

Waking up at very short intervals

A very short time.sleep in a loop keeps the CPU from reaching deep idle states, waking it many times a second. Wait longer or wait on events.

WHY DOES IT WASTE ENERGY?

Every wake-up pulls the CPU out of a deep idle state; dozens of wake-ups a second keep it from reaching low-power states.

WHEN IS IT FINE?

May be necessary where low latency is essential, such as real-time control.

WORK DONE `many wake-ups → few`

WASTEFUL

```
import time


def kuyruk_bekle(kuyruk):
    while kuyruk.bos_mu():
        time.sleep(0.001)
    return kuyruk.al()
```

EFFICIENT

```
def kuyruk_bekle(kuyruk):
    return kuyruk.get()
```

MEASURED time 55.2 ms → 52.9 ms · CPU 805 µs → 247 µs · memory 144 B → 264 B · same result ✓

Both versions wait for work arriving after 50 ms, so the duration is the same. The bad version wakes about 50 times meanwhile; the ratio comes from CPU time.

Source: Linux powertop / Intel: zamanlayıcı uyanmaları · `github.com/fenrus75/powertop`

YK103 info Gives advice 1.23× faster

Starting a process with shell=True

shell=True starts an extra shell process to run the command. Pass the command as a list and use shell=False.

WHY DOES IT WASTE ENERGY?

shell=True first starts /bin/sh, parses the command in the shell and then starts the real program: one extra process.

WHEN IS IT FINE?

Usable when shell features like pipes are really needed; inputs must be trusted.

WORK DONE `2 processes → 1`

WASTEFUL

```
import subprocess


def disk_kullanimi():
    return subprocess.run("df -h", shell=True,
  capture_output=True).stdout
```

EFFICIENT

```
import subprocess


def disk_kullanimi():
    return subprocess.run(["df", "-h"],
  capture_output=True).stdout
```

MEASURED time 14.1 ms → 11.5 ms · memory 82.5 KB → 84.4 KB · same result ✓

The time difference comes from the extra shell process.

Source: Python belgeleri: subprocess (güvenlik ve shell=True) · `docs.python.org/3/library/subprocess.html#security-considerations`

YK104 info Gives advice not measured

Server running in debug mode

debug=True runs a second process for the auto-reloader and keeps watching files. Use debug=False in production.

WHY DOES IT WASTE ENERGY?

The reloader runs a second process and scans files periodically; the debugger adds extra work.

WHEN IS IT FINE?

Fine during development; turn it off in production.

WORK DONE `2 processes + file scanning → 1 process`

WASTEFUL

```
from flask import Flask

app = Flask(__name__)

if __name__ == "__main__":
    app.run(debug=True)
```

EFFICIENT

```
import os

from flask import Flask

app = Flask(__name__)

if __name__ == "__main__":
    app.run(debug=os.getenv("FLASK_DEBUG") == "1")
```

NOT MEASURED A long-running server. The gain is removing the second process and the file-watching load.

Source: Flask belgeleri: Debug Mode · `flask.palletsprojects.com/en/stable/quickstart/#debug-mode`

Measurement method and limits

- Each rule's wasteful (`ornekler/kurallar/YKxxx.py`) and efficient (`iyi/YKxxx.py`) example share a name and run on the same input, generated with the same seed for both.
- Outputs are compared first: the return value or the trace left behind (database, file, screen output). If they differ, nothing is measured.
- Time and CPU time are measured alternately (A, B, A, B…) so momentary machine load affects both sides equally. Each round takes about 60 ms; the result is the median of 5 rounds.
- When an example changes its input, the input is rebuilt before every call and the rebuild is not timed. Caches are cleared before every call.
- Peak memory is measured in a separate run with tracemalloc; only memory allocated during the call counts.
- Database rules were measured on in-memory SQLite (connection and commit rules on an SQLite file). Over a network every query also adds a round trip, so the real gap is usually larger.
- Anyone can repeat it: `python -m greencode kanit` re-runs every measurement. On the website, each rule's page runs the same measurement in the browser.

Rules that show no speed gain

The value of these rules lies in memory, correctness or readability. The measurement shows that openly.

YK020	Reading a whole file with readlines()	132× less memory	Similar time; the real gain is memory: the good version reads the file line by line.
YK036	Query for one row without LIMIT	no difference	An SQLite cursor already stops at the first row, so the difference is negligible. It shows on server databases that prepare the result and send it over the network.
YK049	try/except inside a loop	1.15× slower	Entering a try block has cost nothing since Python 3.11; no difference is expected on the version used here.
YK052	Manual dict comprehension	no difference	
YK062	Needless double conversion	no difference	Too small to measure: the extra copy runs in C and is negligible next to sorting. The gain is simplicity.
YK063	reversed(sorted(...)) or sorted(...)[::-1]	no difference	Too small to measure: the reversed() copy is negligible next to sorting. reverse=True also keeps equal elements in order.
YK065	A list inside sum(), min() or max()	1,089× less memory	A generator takes no memory but can run a little slower than a list comprehension in CPython; the gain is memory.
YK067	len(list(...)) just to count	216× less memory	Similar time; the real gain is memory: lines are not kept in a list while counting.
YK069	Regular expression for plain text	no difference	
YK070	Uncompiled regular expression in a loop	no difference	
YK072	find() just to test containment	no difference	Same speed: both run the same C search. The gain is readability; the Python docs also recommend 'in' for containment.
YK073	Reading a whole file then splitting it into lines	215× less memory	Similar time; the real gain is memory: the good version reads the file line by line.
YK082	Assignment through chained indexing	1.20× slower	This rule is really about correctness: if the chained assignment hits a copy, the change is lost. In pandas 3.0 (copy-on-write) the bad version never changes the table.
YK105	write() one item at a time in a loop	no difference	No difference on a buffered file: write calls already go to an in-memory buffer. The gain is simplicity.
YK111	nunique() to check for a constant series	28.2× less memory	Every value is the same, so both versions scan the whole series and take the same time. The gain is memory: nunique builds a hash table of every value.

Rules not measured, and why

YK005	Busy polling	Waits on an external service (API). The gain is in request count: a loop polling every 0.1 s sends 600 requests a minute, an event-driven design sends one.
YK008	HTTP request without a session in a loop	Needs HTTPS requests to a real server. A session removes the per-request TCP and TLS handshake (usually 1–2 round trips).
YK012	Reloading the same file or model in a loop	Needs a trained model file. The gain is model size × loop length of disk reads and parsing.
YK030	Starting a process in a loop	Needs an external program (ImageMagick). Each process start takes milliseconds; a single process pays that once.

YK033	**objects.create() in a loop (Django)**	Needs a Django project and a database server. The gain is in query count: n INSERTs → 1.
YK034	**save() inside a loop**	Needs a Django project and a database server. The gain is in query count: n UPDATEs → 1.
YK035	**len() on a query result just to count**	Needs a Django project and a database server. The gain is in data moved: n objects → 1 number.
YK075	**DataFrame.append in a loop**	DataFrame.append was removed in pandas 2.0, so the bad version does not run on current pandas. The working form of the same pattern is measured under YK076.
YK089	**Pulling values off the GPU inside a loop**	Needs PyTorch and a GPU. There is no synchronisation wait on the CPU, so the gain only shows on a GPU.
YK090	**Moving the same object to the device in a loop**	Needs PyTorch and a GPU. The gain is removing the CPU→GPU copy on every step.
YK091	**Computing gradients during inference**	Needs PyTorch. The gain depends on the model; the memory kept for gradients during inference disappears entirely.
YK092	**Object-storage request per item in a loop**	Needs a cloud storage (S3) account. The gain comes from overlapping sequential request latencies.
YK093	**One Redis command per iteration**	Needs a Redis server. The gain is in round trips: n → 1.
YK094	**urllib requests in a loop**	Needs requests to a real server. The gain is removing connection setup per request.
YK095	**Synchronous HTTP call inside an async function**	Needs requests to a real server. The gain is not in one call's duration but in other waiting tasks being able to progress.
YK096	**time.sleep inside an async function**	The gain is not in one call's duration but in other tasks on the event loop progressing during the wait.
YK097	**HTTP request without a timeout**	Needs an unresponsive server. A timeout bounds an otherwise infinite wait.
YK104	**Server running in debug mode**	A long-running server. The gain is removing the second process and the file-watching load.